

Node-RED — Operator Manual

What this technology does

Node-RED is the application normalization and persistence layer between ChirpStack application MQTT events and PostgreSQL/TimescaleDB. It validates the decoded application event, derives a stable event identity, writes the canonical uplink plus normalized measurements, and performs the reviewed telemetry/outbox transaction logic.

Node-RED is **not** allowed to declare gateway evidence verified, sign with OpenBao, or submit to Fabric.

Current commissioned topology

```
ULC-03 -> Node-RED A -> ACTIVE
ULC-02 -> Node-RED B -> STAGED / STOPPED / FENCED
```

Only one subscriber/writer is allowed to be active at a time.

Current cloud paths:

```
MQTT:
Node-RED -> mqtt.internal.lorawan.com:18884
          -> node-local HAProxy
          -> Mosquitto mTLS listener :8886

Database:
Node-RED -> pgbouncer.internal.lorawan.com:6432
          -> node-local PgBouncer/HAProxy
          -> current Patroni primary
          -> lorawan_telemetry
```

Application subscription:

```
application/+/device/+/event/up
```

Do not feed gateway Protobuf as923/gateway/.../event/... messages directly into the application-decoding flow.

Rule zero — prove there is only one active writer

Before starting, restarting, or promoting Node-RED, inspect **both** candidates.

On ULC-03 and ULC-02:

```
if [ -d /opt/node-red ]; then
  cd /opt/node-red
  sudo docker compose ps
else
  echo 'NODE_RED_PATH_NOT_FOUND=/opt/node-red'
fi
```

Normal commissioned result: ULC-03 is the active Node-RED A runtime. ULC-02 is staged but not running as an active subscriber/writer.

Hard stop: if both are actively subscribing/writing, do not restart both. Fence one according to the HA runbook and then check for duplicate side effects.

Step 1 — Check the active container and recent logs

On the active host:

```
cd /opt/node-red
sudo docker compose ps
sudo docker compose logs --since=15m --tail=150 node-red
```

If the Compose service name has deliberately changed, discover it with `sudo docker compose config --services` rather than guessing.

PASS means: the active service is running without a restart loop and there is no repeating MQTT, PostgreSQL, flow exception, or credential error.

Step 2 — Verify the MQTT dependency before editing a flow

Confirm the local HAProxy Node-RED route exists:

```
sudo ss -lntp | grep ':18884\b' || true
```

Then use the Mosquitto manual to prove the Node-RED workload identity can subscribe to the intended application topic through the commissioned TLS/mTLS route.

If ChirpStack is publishing application events but Node-RED is not receiving them, inspect:

1. :18884 HAProxy route;
2. Mosquitto :8886 backend;
3. Node-RED MQTT client certificate/identity;
4. ACL/subscription topic;
5. Node-RED connection configuration.

Do not broaden the broker ACL to # merely to make the flow connect.

Step 3 — Verify the database dependency

Confirm local PgBouncer exists:

```
sudo ss -lntp | grep ':6432\b' || true
```

Use the PostgreSQL/PgBouncer manuals to prove a protected `telemetry_writer`-equivalent connection can reach the current Patroni primary through the normal route.

Do not point Node-RED directly at today's PostgreSQL leader.

Step 4 — Verify one recent telemetry row independently of Node-RED

Using a read-only PostgreSQL role, query:

```
SELECT time, dev_eui, metric_name, metric_value, metric_text, metric_bool, unit, quality
FROM telemetry.measurements
ORDER BY time DESC
LIMIT 20;
```

PASS means: recent rows are present after a known approved device event and their values/timestamps are plausible.

The database query is the authoritative persistence check. A green Node-RED node in the editor is not proof that the transaction committed.

Step 5 — Verify stable event identity / duplicate safety

For a known uplink, confirm only one canonical application event row exists for its stable natural/event identity according to the current schema.

If MQTT redelivery creates a second database row, treat that as an idempotency defect. Do not solve it by weakening MQTT QoS or deleting rows manually.

Step 6 — Verify normalized measurements

For the same known event:

1. confirm the canonical `telemetry.uplinks` row exists;
2. confirm expected `telemetry.measurements` rows exist;
3. compare timestamp/device identity/test sequence with the source application event;
4. verify units and quality fields are populated according to the reviewed mapping.

For EMU-01, use the frozen payload-v2 mapping. Do not silently feed SEC-02's different temporary payload format through the EMU-01 decoder.

Step 7 — Understand the Fabric outbox boundary correctly

Node-RED/SQL may create selected durable outbox work according to the reviewed current transaction path, but an outbox row by itself is **not permission to submit to Fabric**.

For the current v2 path, Fabric claim eligibility additionally requires:

```
immutable finalized_payload exists
AND
matching gateway_evidence.event_verification.status = verified
```

Do not manually change pending to another state, fabricate verified, fill `finalized_payload` by reconstructing JSON, or insert a replacement outbox row to make the queue drain.

Use the Fabric Adapter manual to diagnose queue state after the telemetry transaction is proven.

Step 8 — Verify evidence ownership was not crossed

Node-RED must not possess credentials or SQL permissions that let it:

- set verifier-owned evidence rows to verified;

- call OpenBao Transit sign;
- access Fabric client private keys;
- submit Fabric transactions.

If a flow contains any of those authorities, stop and review the deployment because the trust separation has been broken.

Safe flow-change procedure

1. prove which Node-RED candidate is active;
2. keep the standby fenced;
3. export/back up the current approved flow/runtime configuration using the Node-RED runbook;
4. identify the smallest node/function/subflow to change;
5. test with a non-destructive fixture when possible;
6. deploy the change on the active candidate only after syntax/configuration review;
7. observe one known application event;
8. verify the committed database row independently;
9. verify duplicate/idempotency behavior if event identity logic changed;
10. verify evidence/Fabric authority boundaries remain unchanged;
11. update/stage the standby through the active/passive runbook only after the active revision passes.

Do not edit both candidates independently and assume they remain equivalent.

Planned failover rule

Promotion is a **fenced active/passive operation**:

```
prove A is stopped/fenced
-> prove its MQTT session/writer is no longer active
-> start/promote B
-> verify B MQTT + DB path
-> verify one representative real operation
```

Never start B first and then stop A later.

Use the dedicated active/passive HA guide for the exact promotion/failback procedure.

Troubleshooting map

Symptom	First check
ChirpStack event exists, Node-RED sees nothing	MQTT 18884/8886, identity, ACL, topic
Node-RED receives event, no DB row	validation/function error, PgBouncer/DB, transaction rollback
Uplink row exists, measurements absent	normalization/mapping/schema path
duplicate telemetry rows	stable event key + unique constraints + retry path
telemetry exists, outbox absent	current selection/transaction policy; do not fabricate row
outbox exists but not claimed	finalized payload + verifier eligibility + adapter state

Symptom	First check
two writers active	HA fencing failure; stop duplicate authority safely

Completion checklist

- ULC-03 active writer is healthy unless an approved failover deliberately moved ownership;
- ULC-02 standby remains fenced when not owner;
- exactly one active application subscriber/writer exists;
- MQTT route :18884 -> :8886 works;
- database route :6432 works;
- one real approved application event produces one canonical telemetry record;
- normalized measurement rows match the reviewed mapping;
- duplicate delivery does not create duplicate authoritative telemetry;
- Node-RED cannot create verifier authority or Fabric/OpenBao authority.

Detailed guides

- [../server/integrations/node-red/00-README.md](#)
- [../server/integrations/node-red/06-active-passive-ha.md](#)
- [../server/cloud-production/12a-node-red-timescale-telemetry.md](#)

Supporting runbook: 00-README.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

Node-RED Telemetry Pipeline

Use these guides to place Node-RED between ChirpStack application MQTT events and the **logically separate** PostgreSQL/TimescaleDB telemetry database.

There are two deployment profiles:

```
single-host lab
  MQTT host: mosquitto:1883
  DB host:   telemetry-db:5432

three-Droplet cloud HA POC
  Node-RED A: ulc-03 active initially
  Node-RED B: ulc-02 stopped standby
  MQTT host: mqtt.internal.lorawan.com:18884 -> node-local HAProxy -> Mosquitto :8886 pair
  DB host:   pgbouncer.internal.lorawan.com:6432 -> node-local PgBouncer/HAProxy -> Patroni primary
  DB name:   lorawan_telemetry with TimescaleDB enabled
```

The cloud POC does **not** create a separate TimescaleDB server; it keeps `lorawan_telemetry` as a Timescale-enabled logical database inside the Patroni cluster.

Current cloud HA state: active/passive Node-RED server commissioning is complete. Node-RED A on `ulc-03` is healthy/active; Node-RED B on `ulc-02` is fully staged and stopped/fenced. Both candidates map `mqtt.internal.lorawan.com` and `pgbouncer.internal.lorawan.com` to their own host (`10.104.0.8` for A, `10.104.0.4` for B), both private `:18884` MQTT frontends and dedicated `:8886` mTLS identities/ACLs are commissioned, and both carry the same reviewed runtime/flow revision. Keep B stopped until an approved fenced promotion. Real EMU-01 payload-v2 ingestion is now accepted on the production path, including the September 3 post-firewall-fix uplink into TimescaleDB; replay/duplicate safety also has synthetic acceptance evidence. Do not use the obsolete `mqtt-ha.internal.<DOMAIN>:18883` design or the ChirpStack-specific `:18883 -> :8885` route.

Unless a section explicitly says **cloud HA POC**, the older Docker examples assume the single-host lab at `/opt/lorawan-lab`. Perform Node-RED editor actions in a browser connected through the documented SSH tunnel. Commands that begin with `docker compose exec` run a process inside the named container; do not run them on the Raspberry Pi gateway.

Node-RED is responsible for:

- subscribing to decoded ChirpStack uplink events;
- validating identifiers, timestamps, fields, units, and ranges;
- deriving a stable event key for retry-safe storage;
- writing one canonical uplink and its normalized measurements to TimescaleDB;
- preserving the decoded object and raw application data needed for later comparison;
- optionally producing alerts and carefully controlled downlinks.

It is not responsible for LoRaWAN radio configuration, gateway registration, OTAA session management, ChirpStack's internal database, or declaring gateway-integrity evidence *verified*.

For telemetry-attestation-v2, Node-RED is deliberately **not the sole evidence authority**. The independent [Gateway Integrity](#) verifier links the gateway journal to the captured remote gateway MQTT event and accepted ChirpStack application event, runs a pinned trusted decoder outside Node-RED, compares that result with TimescaleDB, and writes the verification state.

Read in this order

1. [01-deploy-node-red.md](#) — Add the persistent Node-RED service and secure the editor.
2. [02-configure-mqtt-and-postgresql.md](#) — Configure application MQTT and the telemetry database connection.
3. [03-build-telemetry-flow.md](#) — Build the validation, deduplication, and TimescaleDB insert flow.
4. [04-automation-and-downlinks.md](#) — Add alerts and gated downlink automation only after ingestion is reliable.
5. [05-testing-and-troubleshooting.md](#) — Test with isolated synthetic input, then accept a real device uplink.
6. [06-active-passive-ha.md](#) — Stage the ulc-02 standby and perform fenced promotion/failback without duplicate subscribers.

Data path

```
LoRaWAN sensor
-> RAK5146 gateway
-> ChirpStack
-> application/+/device+/event/up
-> Node-RED validation and mapping
-> telemetry.uplinks + telemetry.measurements
    |
    +-> Grafana PostgreSQL data source
    |
    +-> gateway-integrity verifier comparison
        -> trusted decoder vs stored normalized values
        -> pending / verified / evidence_gap / integrity_failure
        -> v2 Fabric eligibility only when verified
```

The MQTT topic is case-sensitive. It carries ChirpStack **application** uplinks after network-server processing; gateway event topics are a different Protobuf interface and should not be decoded by this flow.

In the single-host lab, the database target is the Compose service `telemetry-db`. In the cloud HA POC, the target is the logical database `lorawan_telemetry` through local PgBouncer/HAProxy. In both profiles, Node-RED must never write to ChirpStack's `chirpstack` database.

Prerequisites

Complete these first:

- [Register the gateway and verify a real OTAA uplink](#).
- Prepare the TimescaleDB telemetry profile from [TimescaleDB Telemetry Storage](#): single-host lab uses `telemetry-db`; the cloud HA POC enables TimescaleDB in `lorawan_telemetry` on the Patroni cluster.
- For gateway-verified v2 evidence, complete the [Gateway Integrity](#) contracts and reviewed server implementation before enabling v2 Fabric selection.

Deploy [Grafana](#) after Node-RED is writing accepted telemetry. A normal Node-RED database row is application-accepted telemetry; it becomes gateway-verified evidence only after the independent verifier passes.

The primary project mapping is the frozen **EMU-01 Agriculture Kit payload v2**: 46 bytes decoded by ChirpStack into `test_sequence`, `sensor_validity_bitmap`, `soil`, `UV`, `barometer`, both light sensors, environmental values, rain state, and battery. The Node-RED flow preserves the complete decoded object and normalizes those reviewed fields. SEC-02's temporary 6-byte RAK12011 verification payload is a different mapping and must not be silently fed into the EMU-01 flow.

Values needed during setup

Value	Source	Used for
ChirpStack application MQTT topic and credentials	ChirpStack integration and broker ACL	Node-RED MQTT input
Device EUI normalization and stable event-key rule	ChirpStack event contract	Deduplication and database uniqueness
<code>telemetry_writer</code> credentials and the selected DB endpoint (<code>telemetry-db</code> lab or PgBouncer cloud)	TimescaleDB role procedure / cloud HA runbook	Parameterized inserts
Node-RED credential secret and editor password	Generated during Node-RED deployment	Protecting stored credentials and the editor
Device codec fields and units	Verified real uplink or vendor payload specification	Validation and normalized measurements

Do not expose MQTT, PostgreSQL, or either Node-RED editor to the public internet. Bind each editor to loopback, use separate least-privilege MQTT identities per HA candidate, and preserve the **single-active** invariant. Back up the approved Node-RED deployment bundle and TimescaleDB independently; do not use an unsynchronized shared writable Node-RED data directory as an HA mechanism.

Next: [01-deploy-node-red.md](#)

Supporting runbook: 06-active-passive-ha.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

6. Active/Passive Node-RED HA

This guide defines the cloud Node-RED availability contract for the three-Droplet deployment. It does not change the single-host lab profile.

6.1 Goal

Keep exactly one ingestion processor active while maintaining a ready second copy that can take over after the active Node-RED process or host is lost.

```
NORMAL

ulc-03 / 10.104.0.8
Node-RED A = ACTIVE
mqtt.internal.lorawan.com      -> 10.104.0.8:18884
pgbouncer.internal.lorawan.com -> 10.104.0.8:6432

ulc-02 / 10.104.0.4
Node-RED B = STANDBY / container stopped
mqtt.internal.lorawan.com      -> 10.104.0.4:18884
pgbouncer.internal.lorawan.com -> 10.104.0.4:6432

MQTT backends
ulc-01:8886 preferred
ulc-02:8886 backup

Database
local PgBouncer :6432
-> local HAProxy :15432
-> current Patroni primary
```

The **single-active invariant** is mandatory. Never start B while A may still be consuming application/+/device/+/event/up. Ordinary MQTT subscribers would both receive the uplink and create avoidable duplicate work. Database uniqueness remains a safety backstop, not the HA coordination mechanism.

6.2 Why the standby is stopped

A stopped standby is simpler and safer than two always-running Node-RED instances:

- it cannot accidentally subscribe and process the same uplink;
- it does not consume steady-state Node-RED RAM/CPU on ulc-02;
- it can still be pre-staged with the exact image, flows, settings, palette versions, CA files, and protected environment;
- promotion is explicit and auditable.

This is active/passive availability, not active/active load sharing. If automatic election is added later, it must preserve the same fence-before-start rule and be proven under Phase 15 before it is described as automatic HA.

6.3 Files and configuration that must match

Both candidates must use the same reviewed application revision:

```
Node-RED image digest
flows.json / approved flow export
flows_cred.json when used
settings.js
package.json and locked palette versions
LORAWAN_REGION_ID
NODE_RED_CREDENTIAL_SECRET
telemetry schema/event_key contract
```

NODE_RED_CREDENTIAL_SECRET must match if the encrypted credential file is shared between candidates. Store it through the same protected secret-handling procedure on both hosts; never commit it to Git.

Do **not** copy a live `/srv/node-red/data` tree while A is running. Build a versioned deployment bundle from an approved quiescent/exported state, validate its hashes, then install the same bundle on both candidates. Keep host-specific secrets outside that bundle.

6.4 Identities

Use separate MQTT client certificates and keys:

```
Node-RED A / ulc-03: node-red-ingest
Node-RED B / ulc-02: node-red-ingest-standby
```

Both broker ACL entries are read-only:

```
topic read application/+ / device/+ / event/up
```

The two Node-RED broker configurations also need distinct MQTT client IDs. Never copy A's private key onto B merely to make failover easier.

The current `telemetry_writer` database role remains the least-privilege application writer for this POC. Both candidates connect only through their local PgBouncer/HAProxy path with TLS verification.

6.5 Dependency requirement on both hosts

Before claiming the standby is ready, prove:

```
ulc-03:18884 -> ulc-01/02:8886
ulc-02:18884 -> ulc-01/02:8886
ulc-03:6432  -> local :15432 -> Patroni primary
ulc-02:6432  -> local :15432 -> Patroni primary
```

This is why B runs on ulc-02 instead of depending on ulc-03 remotely. Loss of ulc-03 must remove only A, not B's MQTT or database access path.

6.6 Normal state check

Run on both candidates and record the result before any failover test:

```
cd /etc/lorawan-cloud/node-red
sudo docker compose --env-file node-red.env ps node-red
sudo ss -lntp | grep ':1880' || true
```

Expected:

```
ulc-03: node-red running; 127.0.0.1:1880 listening
ulc-02: node-red stopped; no :1880 listener
```

If both are running, stop and investigate immediately. Do not continue with an uplink test in a split-brain state.

6.7 Promotion: A -> B

Use short, sequential gates. Do not batch promotion with unrelated infrastructure changes.

1. Confirm A is failed or deliberately stop A. If the host is only partially reachable, explicitly fence the Node-RED container/service before proceeding.
2. Prove B's local :1880 MQTT route and :6432 database route are healthy.
3. Confirm B has the approved deployment revision and its own MQTT certificate/key.
4. Confirm A is not running.
5. Start B.
6. Verify B's editor remains loopback-only, MQTT connects, and PostgreSQL connects through local PgBouncer.
7. Send a **fresh** staging uplink and prove exactly one canonical telemetry row is created.
8. Record B as the active instance. Do not automatically restart A when its host returns.

Example start on B after the fence gate:

```
cd /etc/lorawan-cloud/node-red
sudo docker compose --env-file node-red.env up -d node-red
sudo docker compose --env-file node-red.env ps node-red
sudo ss -lntp | grep '127.0.0.1:1880'
```

6.8 Failback: B -> A

Failback uses the same stop-before-start rule:

1. Rejoin/recover ulc-03 without starting Node-RED A.
2. Update A to the exact approved flow/image/settings revision currently active on B.
3. Validate A's local MQTT and database routes.
4. Stop B and prove its :1880 listener is gone.
5. Start A.
6. Send one fresh uplink and prove exactly one telemetry row.
7. Return B to stopped standby state.

Do not perform a rolling overlap where both A and B run for convenience.

6.9 QoS and data-loss boundary

The current Node-RED application subscription is QoS 0. Therefore active/passive HA improves recovery from a Node-RED/host outage but **does not guarantee replay of uplinks published while no Node-RED subscriber is active**.

```
A fails
-> promotion window
   -> QoS-0 uplinks may be missed
-> B starts
   -> fresh uplinks resume
```

The two Mosquitto brokers provide backend availability, but this project has not proven replicated MQTT client-session queues between them. Do not claim that a persistent session on one broker is automatically available on the other.

If the final production requirement is zero-loss ingestion, add a durable pre-processing queue or another reviewed at-least-once design and test broker/session failover explicitly. The existing event_key and database unique indexes remain necessary to make later at-least-once delivery retry-safe.

6.10 Phase 15 acceptance

Active/passive Node-RED passes only when all are true:

```
A and B use the same pinned application revision
A and B have separate MQTT private keys/client identities
A and B each have a working local :18884 MQTT route
A and B each have a working local :6432 database route
normal state has exactly one active Node-RED
A failure is fenced before B starts
B processes a fresh post-promotion uplink exactly once
A can rejoin as stopped standby
controlled failback again preserves exactly one active instance
resource test passes on ulc-02 while B is active
promotion gap / any QoS-0 loss is measured and recorded
```

Do not claim automatic failover unless an actual automatic fencing/election mechanism is implemented and tested. Manual fenced promotion is still a valid active/passive topology, but its RTO includes operator detection and promotion time.

Next: return to [Phase 12A](#) for commissioning or [Phase 15](#) for failure acceptance.

Standby secret-group runtime boundary

Each candidate uses a host-local node-red-secrets group for its own MQTT client private key. The Node-RED container receives only that group's numeric GID through Compose group_add; do not add the host login user to the secret group. Keep each key 0640 root:node-red-secrets, its directory 0750 root:node-red-secrets, and the Node-RED data directory numeric 1000:1000 mode 0700. NODE_RED_SECRET_GID is host-specific protected deployment metadata and is not part of the shared flow bundle. The same secret group also protects the candidate-local PgBouncer public-CA copy at /etc/lorawan-pki/node-red-pgbouncer/ca.crt (0640 root:node-red-secrets, directory 0750). Its bytes must match the commissioned /etc/lorawan-pki/pgbouncer/ca.crt on that host; do not grant the container membership in the host postgres group.

Repository-backed shared runtime layer

The canonical shared runtime layer is now deployment/server/integrations/node-red/runtime/. Keep compose.yml, settings.js, package.json, and package-lock.json identical on A and B. The exact dependency lock

SHA-256 is 89289e301cab799ac7e85e2fbe2fc40b34ff195e799313a4f720c642397ba85e. Host-local `node-red.env` supplies the private IP, secret-group GID, unique MQTT client ID, and protected secrets; it is not part of the shared bundle. `flows.json` is now generated from the reviewed telemetry mapping and exact pinned MQTT/TLS/PostgreSQL node contracts. Keep the same flow file on A and B; activation remains gated by dependency/runtime validation and the single-active rule.

Supporting runbook: 12a-node-red-timescale-telemetry.md

This is the original project runbook. Check the current documentation baseline and actual host state before applying historical steps or changing a live system.

12A. Node-RED -> TimescaleDB Telemetry Application Layer

Status: APPLICATION NORMAL PATH COMPLETE / PASS. Node-RED A on u1c-03 is healthy/active, Node-RED B on u1c-02 is staged and deliberately fenced, the single-active invariant is proven, atomic telemetry + Fabric-outbox enqueue is deployed, and both synthetic and real EMU-01 payload-v2 ingestion have been accepted through MQTT -> Node-RED -> TimescaleDB. The September 3 production proof also exposed and corrected the required UFW Docker-bridge allowances for Node-RED's local-private MQTT/PgBouncer paths. The external Fabric handoff is no longer pending: ULC-01 is the commissioned enabled writer and ULC-02 remains write-disabled standby. As of the September 17 research-test diagnosis, recent v2 rows with attempts=0 are not evidence of a disabled Fabric adapter; rows are claim-ineligible when required upstream state such as immutable finalized_payload is absent. Earlier hardware-deferred and pre-Fabric sections below are retained only as chronological commissioning evidence.

12A.1 Goal

Make Node-RED the controlled application transformation layer **before** telemetry reaches the database:

```
LoRaWAN sensor
-> Gateway
-> MQTT
-> ChirpStack
-> application/+/device+/event/up
-> Node-RED
    validate
    normalize
    derive stable event_key
    build parameterized SQL
-> one PostgreSQL statement / transaction
    -> telemetry.uplinks
    -> telemetry.measurements
    -> optional telemetry.fabric_outbox row
-> COMMIT
```

Node-RED is not the database, KMS, Fabric adapter, or cryptographic trust anchor. It must never sign evidence or wait synchronously for Hyperledger Fabric.

12A.2 Preconditions

Before any Node-RED mutation, prove:

```
Phase 9 ChirpStack two-node commissioning = PASS
Phase 10 public ingress normal path = commissioned
Phase 11 physical gateway normal MQTT path = commissioned
Phase 13A database backup + isolated restore rehearsal = PASS
Patroni = 1 primary + 2 streaming replicas
lorawan_telemetry exists with TimescaleDB enabled
telemetry_writer and telemetry_reader authentication paths = PASS
ulc-03 and ulc-02 local PgBouncer :6432 and HAProxy :15432 paths = healthy
one real staging device/gateway can produce an accepted ChirpStack application uplink
```

If the deployment is a migration, Phase 12 must also have made the cloud ChirpStack path authoritative before Node-RED writes production-like telemetry.

12A.3 Commission private Node-RED MQTT routes on both HA candidates

Do not use the obsolete `mqtt-ha.internal.<DOMAIN>` placeholder and do not connect either Node-RED instance directly to one Mosquitto broker. Each Node-RED candidate uses a **node-local** HAProxy :18884 frontend, while HAProxy follows the two dedicated Mosquitto :8886 backends.

The commissioned ChirpStack :18883 frontends on ulc-01/02 terminate at the dedicated username/password :8885 workload listeners and are **not** the Node-RED mTLS route. Keep them unchanged.

Target active/passive topology:

```
Node-RED A ACTIVE on ulc-03
-> mqtt.internal.lorawan.com:18884 -> 10.104.0.8:18884
|
+--> ulc-01 10.104.0.2:8886 preferred
+--> ulc-02 10.104.0.4:8886 backup

Node-RED B STANDBY on ulc-02
-> mqtt.internal.lorawan.com:18884 -> 10.104.0.4:18884
|
+--> ulc-01 10.104.0.2:8886 preferred
+--> ulc-02 10.104.0.4:8886 backup
```

Only **one Node-RED container may be running as the ingestion subscriber at a time**. The passive container stays stopped until the active instance has been fenced or explicitly stopped. This prevents two ordinary MQTT subscribers from processing the same uplink concurrently.

Why :18884: Node-RED needs a stable private frontend on whichever host is active. The dedicated Node-RED-only Mosquitto mTLS listeners remain :8886; gateway-facing :8884 and ChirpStack :8885 remain separate authentication/ACL domains.

Broker identity and ACL

Use a separate MQTT client identity for each Node-RED HA candidate so one host never needs to copy the other host's private key. The current commissioned active identity `node-red-ingest` remains valid for Node-RED A on ulc-03. Before claiming Node-RED HA, issue a second `clientAuth` identity for Node-RED B on ulc-02 and add the same least-privilege read ACL for both identities:

```
user node-red-ingest
topic read application/+device/+event/up

user node-red-ingest-standby
topic read application/+device/+event/up
```

Do not grant either identity gateway-event reads or application/gateway command writes. The two instances must also use distinct MQTT client IDs. Unique identities improve auditability and avoid copying one private key across

hosts; they do **not** permit both Node-RED instances to run simultaneously.

Do **not** change the existing :8884 or :8885 listeners for Node-RED. Create a dedicated :8886 listener with `require_certificate true`, `use_identity_as_username true`, `allow_anonymous false`, and a Node-RED-only ACL. Roll it out one broker at a time: issue and verify the Node-RED client identity first, validate the candidate Mosquitto configuration off-path, activate `ulc-01:8886`, prove the existing :8884 and :8885 listeners remain healthy, then repeat on `ulc-02`. Do **not** deliberately stop a healthy broker to test failover here.

HAProxy shape on each Node-RED candidate host

Use the same frontend/backend policy on both candidate hosts; only the bind address changes:

```
frontend node_red_mqtt_tls
  bind <THIS_NODE_PRIVATE_IP>:18884
  mode tcp
  timeout client 300s
  default_backend node_red_mqtt_brokers

backend node_red_mqtt_brokers
  mode tcp
  timeout server 300s
  option tcp-check
  server mqtt-ulc01 10.104.0.2:8886 check
  server mqtt-ulc02 10.104.0.4:8886 check backup
```

Do not let this MQTT route inherit the global 60s HAProxy client/server idle timeouts. Node-RED uses MQTT keepalive 60; the commissioned route therefore uses explicit 300s frontend/backend timeouts so the proxy does not race the MQTT keepalive interval. Live verification on 2026-09-01 showed the earlier reconnect loop disappear after the scoped override and a fresh client connection; a control `mosquitto_sub` through the same HAProxy + mTLS route completed two clean PINGREQ / PINGRESP cycles.

Candidate addresses are 10.104.0.8 on `ulc-03` and 10.104.0.4 on `ulc-02`. The `ulc-03 :18884` frontend is already commissioned and must be preserved. Before enabling the passive Node-RED, commission the equivalent private 10.104.0.4:18884 frontend on `ulc-02` and validate it with `haproxy -c` before reload.

After each frontend is commissioned, prove TLS with SNI/hostname `mqtt.internal.lorawan.com` and that the host-specific Node-RED identity can read `application/+device/+event/up` while a forbidden command publication is denied.

No public Node-RED MQTT listener is created on either host.

12A.4 Database route from Node-RED

Each Node-RED candidate uses the PgBouncer endpoint on its **own host**:

```
Node-RED A on ulc-03
-> pgbouncer.internal.lorawan.com:6432 -> 10.104.0.8:6432
-> local HAProxy :15432 -> current Patroni primary -> lorawan_telemetry

Node-RED B on ulc-02
-> pgbouncer.internal.lorawan.com:6432 -> 10.104.0.4:6432
-> local HAProxy :15432 -> current Patroni primary -> lorawan_telemetry
```

This is critical: the standby must not depend on `ulc-03` for either MQTT or PostgreSQL access, otherwise loss of `ulc-03` would also remove its dependencies.

Use `telemetry_writer` only. Keep hostname/CA verification enabled. Do not use `telemetry_admin`, `postgres`, or the `ChirpStack` role.

Before starting Node-RED, prove a rollback-only insert/duplicate test through this exact path using the procedures in:

- `../integrations/timescaledb/02-create-telemetry-schema.md`
- `../integrations/timescaledb/03-connect-and-verify.md`

The required Timescale objects are:

```
telemetry.uplinks      hypertable
telemetry.measurements hypertable
telemetry.device_registry
telemetry.latest_uplinks
telemetry.latest_measurements
```

`telemetry.fabric_outbox` is an ordinary PostgreSQL table and is created/protected before Fabric selection is enabled.

12A.5 Deploy Node-RED active/passive on ulc-03 and ulc-02

Use the cloud profile in:

- `../integrations/node-red/01-deploy-node-red.md`
- `../integrations/node-red/02-configure-mqtt-and-postgresql.md`
- `../integrations/node-red/03-build-telemetry-flow.md`

Required cloud substitutions:

```
Node-RED A:      ulc-03 / 10.104.0.8 / ACTIVE initially
Node-RED B:      ulc-02 / 10.104.0.4 / STANDBY initially
Editor listener:  127.0.0.1:1880 only on each host
MQTT host:       mqtt.internal.lorawan.com
MQTT port:       18884 on the local host
MQTT TLS:        mTLS, hostname/CA verification enabled
MQTT topic:      application/+/device+/event/up
Database host:   pgbouncer.internal.lorawan.com
Database port:   6432 on the local host
Database:        lorawan_telemetry
Database role:   telemetry_writer
```

Per-host container mappings:

```
ulc-03 Node-RED A:
  mqtt.internal.lorawan.com    -> 10.104.0.8
  pgbouncer.internal.lorawan.com -> 10.104.0.8

ulc-02 Node-RED B:
  mqtt.internal.lorawan.com    -> 10.104.0.4
  pgbouncer.internal.lorawan.com -> 10.104.0.4
```

Pin the exact same Node-RED image digest, PostgreSQL palette versions, approved flows, `settings.js`, region value, and credential-encryption secret on both candidates. Do **not** copy a live `/srv/node-red/data` directory while the active instance is writing it; distribute a reviewed versioned deployment bundle plus separately protected secrets. Keep the standby container stopped after validation.

Protect database/MQTT credentials outside Git. Require authenticated editor access through an SSH tunnel on whichever instance is active; never publish either editor publicly as a troubleshooting shortcut. See `../integrations/node-red/06-active-passive-ha.md` for promotion/failback.

12A.6 Prove MQTT before database writes

Deploy only:

```
mqtt in -> json -> debug
```

Send one real staging-device uplink through the normal gateway/ChirpStack path.

For the EMU-01 payload-v2 baseline, require the expected application event to contain the reviewed fields such as:

```
payload_version = 2
test_sequence
sensor_validity_bitmap
approved decoded sensor values
```

The MQTT node being “connected” is not enough. A real current application event must arrive.

12A.7 Enable validation/normalization and atomic telemetry writes

Use the reviewed function/SQL in ../integrations/node-red/03-build-telemetry-flow.md.

The order is:

```
ChirpStack event
-> validate DevEUI/time/payload version
-> normalize approved decoded values/units
-> derive stable event_key
-> parameterized SQL
-> INSERT uplink
-> INSERT normalized measurements
-> COMMIT
```

Invalid sensor-validity groups must not be mislabeled as measured stale values. Preserve the complete decoded object in `payload_json` while normalized metrics go into `telemetry.measurements`.

Do not decode the LoRaWAN binary payload a second time in Node-RED when ChirpStack has already run the approved codec.

12A.8 Prove retry safety before Fabric selection

Deliberately replay the **same application event** through the Node-RED flow without changing its stable identity.

Pass when database uniqueness rules leave:

```
exactly one canonical telemetry.uplinks row
exactly one row per normalized metric identity
no second canonical application record
```

This is an application idempotency verification, not a service-failure injection.

12A.9 Create and protect the Fabric outbox, then enable one selected event

Before Node-RED can enqueue Fabric work, complete the outbox schema/permissions/immutability portion of:

../fabric-attestation/02-create-outbox-and-adapter.md

Node-RED may then add the reviewed outbox CTE to the **same PostgreSQL statement**:

```
BEGIN-equivalent single statement
telemetry.uplinks
telemetry.measurements
selected telemetry.fabric_outbox
COMMIT
```

For the first commissioning proof, select only the approved staging device/event. Do not send every sample to Fabric merely because the queue exists.

Node-RED stores only the source identity and queue request. It does **not** create `canonical_json`, calculate the production evidence digest, sign with OpenBao, or mark Fabric confirmation.

12A.10 Normal-path pass condition

Phase 12A passes only when all are true:

```
private Node-RED MQTT HAProxy :18884 exists on both ulc-03 and ulc-02
both Node-RED mTLS identities can read only approved application uplinks
exactly one Node-RED ingestion container is active; the other is stopped
both Node-RED editors are loopback-only and the active editor is authenticated
telemetry_writer connects through the local PgBouncer/HAProxy path on both candidates with TLS verification
one real staging uplink reaches Node-RED
Node-RED validates/normalizes the event before database storage
one canonical uplink + reviewed measurements are stored
replaying the same event does not create duplicates
one selected event can atomically create a fabric_outbox row
Fabric/KMS are not called synchronously from Node-RED
```

Do **not** inject failures during the initial Phase 12A commissioning. After both Node-RED candidates are staged and the single-active invariant is proven, Node-RED promotion/failback and host-loss behavior are tested in Phase 15. Mosquitto, Patroni, PgBouncer, HAProxy, and gateway failure cases remain Phase 15 work.

Next setup phase: [14a-grafana-cloud-deployment.md](#).

Recorded server-only Phase 12A preparation boundary - 2026-08-27

The database-side Fabric outbox prerequisite is now **COMPLETE / PASS**: `telemetry.fabric_outbox` is commissioned on the Patroni cluster, replicated 3/3, protected by the documented ACL and immutability rules, and contains zero commissioning rows. Node-RED outbox enqueue remains disabled.

The physical gateway is temporarily unavailable, so Phase 12A cannot yet claim its full normal-path preconditions or application-ingestion PASS. While that hardware dependency is deferred, only gateway-independent server preparation is allowed. The next checkpoint is a **read-only live-state preflight** from `ulc-03` that inventories: current Mosquitto :8884 authentication/ACL directives on both brokers, MQTT PKI material needed for a future Node-RED client identity, `ulc-03` HAProxy and target :18884 availability, local PgBouncer/HAProxy database routes, and the absence/current state of Node-RED runtime files/listeners.

Do not assume the planning text describing :8884 mTLS matches the current live broker state: earlier Phase 9 evidence observed `require_certificate false`. Do not add the Node-RED ACL, issue a client certificate, reload Mosquitto, add the :18884 HAProxy frontend, or start Node-RED until the live preflight is reviewed. The repository also still contains `nodered/node-red@sha256:10f40d0a83e7e5852b13d4d472b2006b05b1cca6d55e2f29a55a12c25a630cb6` rather than an approved image digest, so Node-RED deployment remains blocked on an explicit image pin in addition to the later real-uplink acceptance dependency.

Recorded server-only preflight harness note - 2026-08-27

The first read-only server-side preflight stopped after proving ulc-03 HAProxy/PgBouncer listeners because the operator-side openssl s_client attempted to read /etc/lorawan-pki/pgbouncer/ca.crt without privilege. This path is intentionally protected by the commissioned PgBouncer PKI boundary (750 root:postgres directory, 640 root:postgres files). Do **not** loosen ownership or mode for a diagnostic. No Phase 12A service/configuration mutation occurred in this failed harness attempt.

A follow-up sudo openssl s_client then reached 10.104.0.8:6432 but returned ssl3_get_record:wrong version number. This is a second **client harness/protocol mismatch**, not evidence that PgBouncer TLS is broken. PostgreSQL and PgBouncer negotiate TLS only after the client sends the PostgreSQL SSLRequest packet; raw TLS from byte zero is invalid for this port. The corrected OpenSSL probe must therefore use both privileged CA access and PostgreSQL STARTTLS negotiation: sudo openssl s_client -starttls postgres ... -CAfile /etc/lorawan-pki/pgbouncer/ca.crt -verify_hostname pgbouncer.internal.lorawan.com. A real psql client with sslmode=verify-full, logical host=pgbouncer.internal.lorawan.com, and physical hostaddr=10.104.0.8 remains the application-level proof. Do not change PgBouncer, HAProxy, PKI ownership/mode, or TLS settings based on either harness failure.

Recorded PgBouncer TLS preflight PASS - 2026-08-27

The corrected ulc-03 host probe is now **PASS**. Use sudo openssl s_client -starttls postgres for PgBouncer TLS checks: sudo is required to traverse/read the commissioned 750 root:postgres / 640 root:postgres PKI boundary, and -starttls postgres is required because PgBouncer/PostgreSQL expects an SSLRequest before the TLS handshake. The live probe to 10.104.0.8:6432 negotiated TLS 1.3 with TLS_AES_256_GCM_SHA384, verified the CA, and verified peer name pgbouncer.internal.lorawan.com. Keep the PKI permissions unchanged. The next server-only Phase 12A checkpoint is the read-only :8884 Mosquitto authentication/ACL inventory on both brokers; do not issue the Node-RED certificate or reload Mosquitto until that live state is reviewed.

Recorded ulc-01 :8884 live-auth partial inventory - 2026-08-27

The first Phase 12A MQTT-auth inventory reached ulc-01 and captured the currently loaded listener-specific configuration before its value-summary helper aborted. per_listener_settings true is active. The dedicated ChirpStack listener remains 10.104.0.2:8885 with its password/ACL files. The separate gateway-facing listener in tls.conf is :8884 with the commissioned MQTT CA/server certificate/key, TLS 1.3, require_certificate false, and allow_anonymous false. No use_identity_as_username directive was observed in the active files shown before the stop. Therefore the live ulc-01:8884 listener is **not currently mTLS/client-certificate-authenticated**, and the earlier Phase 12A planning text that described :8884 as an already-existing mTLS backend must not be treated as live-state evidence. Historical Phase 9 runtime proof remains relevant: unauthenticated MQTT CONNECT was rejected on both :8884 brokers with CONNACK 0x05 while server TLS/hostname verification passed.

The inventory wrapper exited only because set -euo pipefail treated a no-match grep for an optional directive such as use_identity_as_username as fatal inside command substitution. This is a read-only harness defect. No Mosquitto, HAProxy, PKI, or Node-RED mutation occurred. Continue with a corrected listener-specific parser that tolerates absent optional directives and capture ulc-02 before designing any Node-RED client-certificate transition.

Recorded Node-RED MQTT listener design decision - 2026-08-27

The corrected listener-specific inventory completed on both brokers with identical live results. per_listener_settings=true; the existing :8884 listener is TLS 1.3 with require_certificate=false, allow_anonymous=false, no use_identity_as_username, no listener-local password/ACL/plugin, and a verified mqtt.internal.lorawan.com server certificate. A TLS handshake without a client certificate succeeds on both brokers. Historical Phase 9 MQTT CONNECT evidence still proves anonymous MQTT is rejected with CONNACK 0x05; therefore :8884 is server-TLS with anonymous authorization denied, **not mTLS**. :8886 is unused on both brokers. No runtime mutation occurred during this inventory.

Decision: preserve existing :8884 and ChirpStack :8885 exactly. Phase 12A will use a dedicated Node-RED authentication domain: ulc-03 HAProxy :18884 -> ulc-01/ulc-02 Mosquitto :8886. The :8886 listener requires a clientAuth Node-RED certificate, maps certificate CN to MQTT username, denies anonymous access, and uses a Node-RED-only ACL permitting only topic read application/+device/+event/up. The candidate identity name is node-red-ingest. The Node-RED client private key lives only on the Node-RED side; brokers receive no client private key and the CA private key remains only in its existing root-controlled issuance boundary on ulc-03. When provider firewalls are enforced, permit 8886/tcp only from the ulc-03/Node-RED path to ulc-01/02; never expose 8886 publicly.

Next boundary: read-only PKI issuance preflight on ulc-03. Prove the commissioned CA certificate/key match without printing key material, prove the node-red-ingest identity is not already issued/installed, and preserve the current CA serial state. Only after that passes may a new clientAuth identity be issued.

Recorded Node-RED MQTT PKI preflight broker-check harness stop - 2026-08-27

The first Node-RED MQTT client-PKI preflight completed steps 1 through 8 successfully on ulc-03: the commissioned CA boundary remained 0700 root:root with ca.crt 0644 root:root and ca.key 0600 root:root; CA certificate/key public-key hashes matched; the local MQTT CA copy matched SHA-256 6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb; the CA serial-file baseline was captured; node-red-ingest was not already issued; and /etc/lorawan-pki/node-red-mqtt contained no prior client identity. The run then stopped on the first broker trust check because the SSH command over-escaped the remote awk '{print \$1}' expression, so remote awk received a literal backslash before \$1 and exited with backslash not last character on line.

Treat this as a **read-only harness defect after eight PASS gates**, not as a broker CA mismatch or :8886 conflict. No certificate, key, Mosquitto file, HAProxy file, firewall rule, or Node-RED runtime was created or changed. Do not repeat the already-proven CA/identity checks merely because this wrapper stopped. Continue only with broker hostname verification, remote MQTT-CA SHA comparison, and :8886 listener absence on ulc-01 and ulc-02. Avoid nested remote awk quoting: return raw sha256sum / ss output over SSH and perform parsing locally on ulc-03.

Recorded Node-RED MQTT PKI preflight PASS - 2026-08-27

The complete read-only preflight is **PASS**. The commissioned CA root/key boundary on ulc-03 is unchanged; the CA certificate and private key public-key hashes match; the ulc-03 MQTT CA copy matches the issuing CA; the CA serial-file baseline was recorded without mutation; CN node-red-ingest is unused; /etc/lorawan-pki/node-red-mqtt contains no prior client identity; both brokers trust the same CA; :8886 is free on both brokers; and existing :8884/:8885 listeners remained present. Proceed with one bounded client-certificate issuance only. Do not change Mosquitto, HAProxy, or Node-RED in the same step.

Recorded Node-RED MQTT client certificate issuance - 2026-08-27

The dedicated Node-RED MQTT workload identity is now issued and validated under /root/lorawan-pg-ca/node-red-ingest-issuance-20260827T142128Z on ulc-03. Its Common Name is exactly node-red-ingest; it is RSA-3072, grants clientAuth only, does not grant serverAuth, chains to the commissioned internal CA, and matches its private key. The CA serial file remained byte-identical because issuance used an explicit random serial. All issuance files remain root-only and **no client private key has been transferred or installed into the Node-RED runtime yet**.

The next mutation is intentionally broker-local and canary-first. On ulc-01 only, reserve /etc/mosquitto/node-red.acl for the exact node-red-ingest read-only policy and /etc/mosquitto/conf.d/node-red.conf for the dedicated 10.104.0.2:8886 mTLS listener. Before any live restart, validate a disposable loopback copy of the full three-listener policy (28884, 28885, 28886) while the production Mosquitto service remains active and live :8886 remains absent. Only after that off-path validation passes may the ulc-01 live canary be activated. ulc-02, HAProxy :18884, and the Node-RED runtime remain untouched during this preparation boundary.

Listener ownership clarification - 2026-08-28

The instruction in this phase to preserve :8884 unchanged applies to **Node-RED commissioning**: Node-RED must not repurpose the gateway authentication domain and instead uses dedicated :8886. It does not mean :8884 should remain server-TLS-only forever. Gateway commissioning in Phase 10/11 is responsible for hardening the gateway-facing :8884 listener to client-certificate authentication with the exact Gateway-EUI ACL. Keep :8885 for ChirpStack and :8886 for Node-RED separate.

Current Node-RED MQTT broker commissioning status - 2026-08-28

The dedicated Node-RED private MQTT backend is now **COMPLETE / PASS on both brokers**.

ulc-01 pre-mutation verification proved gateway mTLS :8884 and ChirpStack 10.104.0.2:8885 were live, :8886 was unused, and the active broker TLS paths were /etc/lorawan-pki/mqtt/ca.crt, /etc/lorawan-pki/mqtt/server.crt, and /etc/lorawan-pki/mqtt/server.key. A disposable loopback Mosquitto instance validated the new Node-RED listener policy before the live change. The live 10.104.0.2:8886 listener was then installed with TLS 1.3, require_certificate true, use_identity_as_username true, allow_anonymous false, and /etc/mosquitto/node-red.acl. Existing :8884 and :8885 remained healthy after restart.

Direct client verification from ulc-03 using the already-issued CN = node-red-ingest identity proved the ulc-01:8886 TLS boundary with hostname mqtt.internal.lorawan.com (Verification: OK, Verify return code: 0 (ok)). The first MQTT ACL probe was inconclusive only because the client output was filtered; a later retry initially returned 127 because mosquitto_sub and mosquitto_pub were not installed on ulc-03. After installing the mosquitto-clients utilities, the authoritative ACL proof passed: subscription to application/+device/+event/up returned CONNACK (0), SUBACK, and granted QoS 0; the final wait returned RC=27 only because no live uplink arrived during the two-second window. An intentionally forbidden QoS1 publish to application/test/device/test/command/down returned MQTT v5 PUBACK RC:135 and Not authorized.

ulc-02 then passed the streamlined pre-mutation gate with gateway :8884 and ChirpStack 10.104.0.4:8885 live and :8886 unused. The same dedicated Node-RED configuration was installed at 10.104.0.4:8886; Mosquitto restarted successfully and all three listeners remained present. Direct ACL verification from ulc-03 matched ulc-01: allowed application-uplink subscription returned CONNACK (0), SUBACK, granted QoS 0, and only the expected no-message timeout RC=27; the forbidden command publish returned PUBACK RC:135 / Not authorized.

Authoritative broker state:

ulc-01 10.104.0.2:8884	gateway mTLS	PASS
ulc-01 10.104.0.2:8885	ChirpStack	PASS
ulc-01 10.104.0.2:8886	Node-RED mTLS	PASS
ulc-02 10.104.0.4:8884	gateway mTLS	PASS
ulc-02 10.104.0.4:8885	ChirpStack	PASS
ulc-02 10.104.0.4:8886	Node-RED mTLS	PASS
Node-RED allowed read ACL		PASS
Node-RED forbidden write denial		PASS

Do not repeat the broker listener, TLS, or ACL tests unless a later failure provides a reason. The next provider-independent boundary is the private ulc-03 HAProxy frontend 10.104.0.8:18884, routing to ulc-01:8886 as preferred and ulc-02:8886 as backup. No public listener is required for this Node-RED path.

ulc-03 Node-RED HAProxy pre-mutation gate PASS - 2026-08-28

Live ulc-03 pre-mutation verification passed: HAProxy is active; 10.104.0.8:18884 is unused; no existing node_red_mqtt / :18884 / :8886 HAProxy stanza exists; and TCP reachability from ulc-03 to both commissioned Node-RED MQTT backends succeeded (10.104.0.2:8886 and 10.104.0.4:8886, exit 0). The next bounded mutation is to append only the documented node_red_mqtt_tls frontend and node_red_mqtt_brokers backend, validate /etc/haproxy/haproxy.cfg with haproxy -c, then reload HAProxy.

ulc-03 Node-RED HAProxy activation PASS - 2026-08-28

The private Node-RED MQTT frontend is now active on ulc-03 at 10.104.0.8:18884. Before reload, /etc/haproxy/haproxy.cfg was preserved at /etc/haproxy/haproxy.cfg.before-node-red-20260828T043108Z; the new node_red_mqtt_tls frontend and node_red_mqtt_brokers backend passed haproxy -c -v. HAProxy reloaded successfully, remained active, bound 10.104.0.8:18884, and accepted a TCP connection on that frontend. The backend policy is ulc-01 10.104.0.2:8886 preferred with ulc-02 10.104.0.4:8886 as backup. NODE_RED_HAPROXY_CONFIG=PASS and NODE_RED_HAPROXY_18884=PASS are authoritative. The next boundary is one end-to-end Node-RED client mTLS/ACL verification through 10.104.0.8:18884; do not repeat direct :8886 broker tests unless that path fails.

Node-RED active/passive HA decision - 2026-08-28

The earlier standalone-Node-RED decision is superseded. The selected production/POC target is **active/passive Node-RED**: Node-RED A on ulc-03 is active initially and Node-RED B on ulc-02 is a pre-staged stopped standby. Never run both ordinary application-uplink subscribers at the same time. Promotion must fence or stop the old active instance before starting the standby; failback follows the same stop-before-start rule.

Each candidate must use its own local dependency routes (:18884 MQTT HAProxy and :6432 PgBouncer) so loss of ulc-03 does not strand the standby. The image digest, flows, palette versions, settings, region configuration, and credential-encryption secret must match. MQTT private keys are **not** shared: the active keeps node-red-ingest, while the standby receives a separately issued clientAuth identity with the same read-only topic ACL.

This HA change improves service availability but does **not** by itself guarantee zero telemetry loss during promotion. The current application subscription is QoS 0 and the two Mosquitto brokers do not have a proven replicated client-session queue. Phase 15 must measure the promotion gap and prove a fresh post-promotion uplink is processed exactly once. If the final production requirement is zero-loss ingestion, add and prove a durable pre-Node-RED queue/at-least-once design separately rather than claiming active/passive alone provides replay.

Current runtime evidence remains historical and valid: the ulc-03 10.104.0.8:18884 frontend is commissioned, both :8886 brokers are commissioned, and the original node-red-ingest certificate is issued. **Node-RED HA is not yet commissioned** until ulc-02 :18884, the standby MQTT identity/ACL, the standby runtime bundle, and the single-active promotion/failback test all pass.

ulc-02 Node-RED standby HAProxy pre-mutation gate PASS - 2026-08-28

Live ulc-02 pre-mutation verification passed for the active/passive standby path: HAProxy is active and its current configuration validates; 10.104.0.4:18884 is unused; no existing node_red_mqtt / :18884 stanza exists; and both commissioned Node-RED MQTT backends are reachable from ulc-02 (10.104.0.2:8886 and 10.104.0.4:8886, exit 0). The next bounded mutation is to add only the local standby-host 10.104.0.4:18884 TCP frontend with ulc-01:8886 preferred and ulc-02:8886 backup, validate /etc/haproxy/haproxy.cfg before reload, then prove the local listener. Do not start Node-RED B in this step.

ulc-02 Node-RED standby HAProxy activation PASS - 2026-08-28

The standby host now has its own private Node-RED MQTT frontend at 10.104.0.4:18884. The candidate HAProxy configuration validated before reload; HAProxy reloaded cleanly, remained active, bound 10.104.0.4:18884, and accepted a TCP connection on that frontend. Rollback copy: /etc/haproxy/haproxy.cfg.before-node-red-standby-20260828T044807Z. Backend order matches the active host: 10.104.0.2:8886 preferred and 10.104.0.4:8886 backup. ULC02_NODE_RED_HAPROXY_CONFIG=PASS and ULC02_NODE_RED_HAPROXY_18884=PASS are authoritative. Node-RED B remains unstaged/stopped.

The selected standby MQTT identity is now frozen as node-red-ingest-standby. It must receive its own clientAuth certificate/private key and the same read-only application/+/device/+/event/up ACL as node-red-ingest; do not

reuse Node-RED A's private key. The next boundary is a read-only PKI identity-availability gate on ulc-03, followed by one bounded certificate issuance if unused.

Standby Node-RED PKI preflight wrong-host stop - 2026-08-28

The first standby identity preflight was accidentally run on ulc-02 instead of the CA workstation ulc-03. The wrapper printed only `=== STANDBY NODE-RED PKI GATE ===` and then exited under `set -e` at the first CA-file existence check because `/root/lorawan-pg-ca` is intentionally confined to ulc-03. Treat this as a harmless wrong-host stop: no certificate, key, CA serial, Mosquitto ACL, HAProxy configuration, or Node-RED runtime changed. Rerun the same read-only identity/serial gate on ulc-03 only.

Standby Node-RED MQTT PKI preflight PASS - 2026-08-28

The read-only standby identity gate was rerun on the correct CA host ulc-03 and passed. `/root/lorawan-pg-ca/ca.ca.srl` still has SHA-256 `50df8c462ef9465ab9198284fa1234f0cbfa4f33eb9779ce6d50dd23a618463d`, no `node-red-ingest-standby` issuance directory exists, and no existing certificate with CN = `node-red-ingest-standby` was found under the CA boundary. `CA_SERIAL_BASELINE=PASS`, `NODE_RED_STANDBY_IDENTITY_UNUSED=PASS`, and `NODE_RED_STANDBY_PKI_GATE=PASS` are authoritative. The next bounded mutation is issuance of exactly one RSA-3072, clientAuth-only certificate for `node-red-ingest-standby` with an explicit random serial; do not alter Mosquitto, HAProxy, or either Node-RED runtime in the issuance step.

Standby Node-RED MQTT client certificate issuance PASS - 2026-08-28

The separate standby MQTT workload identity is now issued and validated on ulc-03 under `/root/lorawan-pg-ca/node-red-ingest-standby-issuance-20260828T045539Z`. The certificate subject is CN = `node-red-ingest-standby`, issuer CN = `LoRaWAN PostgreSQL Internal CA`, serial `A80E60B0234A7ADEE76925F73B6FDCBF`, validity `2026-08-28 04:55:40Z through 2027-09-29 04:55:40Z`, SHA-256 fingerprint `2B:C4:A2:7A:42:57:E4:E4:15:BC:CA:0E:1E:8C:17:7D:1D:EA:F5:21:97:43:3F:D5:3A:ED:C3:DB:F9:D0:DF:AC`, certificate SHA-256 `f3d251a6abce3e20bc99bfe456fb5f7c3cf071acded4553bdc6a76d42fd82fbd`, and public-key SHA-256 `0d0a041770d56883e4c335ed4691cc2ae8c279f03e672ca4f5f6e0755888d4f1`. The RSA-3072 certificate/key public keys match, `openssl verify -purpose sslclient` passed, `sslserver` purpose was rejected, and the CA serial-file SHA-256 remained `50df8c462ef9465ab9198284fa1234f0cbfa4f33eb9779ce6d50dd23a618463d` because issuance used an explicit random serial. `NODE_RED_STANDBY_CLIENT_CERT_ISSUANCE=PASS` is authoritative. The standby private key remains root-only on ulc-03; do not copy it to a broker or to Node-RED A.

Next boundary: extend the dedicated Node-RED :8886 ACL on ulc-01 only with a second read-only identity, `node-red-ingest-standby`, preserving the existing `node-red-ingest` entry and all :8884/:8885 listeners. Verify ulc-01 before mirroring the ACL to ulc-02.

ulc-01 standby Node-RED MQTT ACL activation PASS - 2026-08-28

The ulc-01:8886 Node-RED ACL now authorizes both dedicated read-only client identities: `node-red-ingest` and `node-red-ingest-standby`, each only for `application/+device/+event/up`. Before the change, the original `node-red-ingest` ACL entry was re-proven. A timestamped rollback copy was created at `/etc/mosquitto/node-red.acl.before-standby-20260828T045709Z`; Mosquitto restarted successfully; and gateway :8884, ChirpStack 10.104.0.2:8885, and Node-RED 10.104.0.2:8886 all remained listening. `ULC01_NODE_RED_STANDBY_ACL=PASS` is authoritative. Next prove the newly issued `node-red-ingest-standby` certificate can subscribe but cannot publish through the ulc-01 canary before changing the ulc-02 ACL.

ulc-01 standby Node-RED MQTT identity authorization PASS - 2026-08-28

The separately issued CN = `node-red-ingest-standby` client identity was tested directly against the commissioned ulc-01 10.104.0.2:8886 Node-RED mTLS listener. The allowed subscription to `application/+device/+event/up` completed MQTT CONNECT and SUBACK successfully with granted QoS 0; the final `ALLOWED_READ_RC=27` was only the expected two-second no-message timeout. An intentionally forbidden QoS1 publish to

application/test/device/test/command/down returned MQTT v5 PUBACK RC:135 / Not authorized; the client process exit code of 0 does not override the broker denial. Therefore the standby identity has the intended read-only authorization on the ulc-01 canary. Next mirror the same two-identity ACL to ulc-02, preserve listeners :8884, :8885, and :8886, then repeat only the standby-identity ACL proof through ulc-02.

ulc-02 standby Node-RED MQTT ACL activation PASS - 2026-08-28

The standby identity ACL was mirrored to ulc-02 successfully. Before mutation, /etc/mosquitto/node-red.ac1 contained the existing active identity node-red-ingest and the read-only application-uplink topic permission. The ACL was backed up at /etc/mosquitto/node-red.ac1.before-standby-20260828T050127Z, rewritten to the canonical two-identity policy for node-red-ingest and node-red-ingest-standby, and Mosquitto restarted successfully. mosquitto remained active and all commissioned listeners remained present: gateway :8884, ChirpStack 10.104.0.4:8885, and Node-RED mTLS 10.104.0.4:8886. ULC02_NODE_RED_STANDBY_ACL=PASS is authoritative. The next boundary is one end-to-end authorization test through the standby host's local HAProxy frontend 10.104.0.4:18884 using the node-red-ingest-standby certificate; Node-RED B must remain stopped.

Node-RED B full MQTT path PASS - 2026-08-28

The standby identity node-red-ingest-standby was proven end-to-end through the standby host's private HAProxy frontend 10.104.0.4:18884. TLS 1.3 completed successfully and verified peer name mqtt.internal.lorawan.com; MQTT CONNECT returned CONNACK 0; subscription to application+/device+/event/up returned SUBACK with QoS 0 and only the expected no-message timeout RC=27; an intentionally forbidden QoS1 command publication returned MQTT v5 PUBACK reason code 135 / Not authorized. This closes the Node-RED B MQTT dependency path. The next boundary is the standby database path through local ulc-02:6432 PgBouncer/HAProxy; do not start Node-RED B yet.

Node-RED B database rollback harness credential lookup stop - 2026-08-28

The first rollback-only telemetry_writer test from ulc-02 reached 10.104.0.4:6432 but psql prompted for a password and then stopped with fe_sendauth: no password supplied. This is a client harness mismatch, not a PgBouncer or database failure: the temporary .pgpass entry used host 10.104.0.4, while the libpq connection string uses logical host=pgbouncer.internal.lorawan.com with hostaddr=10.104.0.4. .pgpass matching is performed against the logical host, so the password entry did not match. No SQL transaction began and no telemetry row was created. Correct only the .pgpass host field to pgbouncer.internal.lorawan.com, preserve hostaddr=10.104.0.4 for routing and sslmode=verify-full for hostname verification, then rerun the same rollback-only write/idempotency test.

Node-RED B database authentication PASS - 2026-08-28

The corrected auth-only probe from ulc-02 succeeded through the exact standby database endpoint pgbouncer.internal.lorawan.com with physical hostaddr=10.104.0.4, port 6432, sslmode=verify-full, and role telemetry_writer. The session reached database lorawan_telemetry, backend address 10.104.0.2, and returned pg_is_in_recovery() = false, proving the local ulc-02 PgBouncer/HAProxy path follows the current writable Patroni primary. The earlier password failure was only a .pgpass hostname mismatch. Next run one rollback-only insert/idempotency transaction through the same endpoint; do not start Node-RED B yet.

Node-RED B database rollback SQL-variable harness stop - 2026-08-28

The rollback-only Node-RED B database write/idempotency test authenticated successfully but stopped before any INSERT because the SQL sent via this psql -c invocation still contained the literal token : 'testkey'. PostgreSQL therefore returned a syntax error at : on the first VALUES row. No transaction body executed and no telemetry row was created. Treat this as a client harness defect only; the previously proven telemetry_writer -> ulc-02:6432 -> writable Patroni leader path remains authoritative PASS. Correct the retry by expanding a shell-generated

safe test key directly into the SQL text, then keep the entire write/idempotency proof inside one transaction followed by ROLLBACK.

Node-RED B database write and idempotency PASS - 2026-08-28

The standby database dependency is now complete. From `u1c-02`, `telemetry_writer` authenticated through local PgBouncer `10.104.0.4:6432` using verify-full TLS, then a rollback-only transaction inserted one row into `telemetry.uplinks` and one row into `telemetry.measurements`. Repeating both inserts with the same event identity returned `INSERT 0 0`, and the in-transaction counts were exactly `uplink_count=1` and `measurement_count=1`, proving the unique/idempotency constraints collapse duplicate retries. The transaction was rolled back and post-rollback residue counts were exactly zero for both tables. `NODE_RED_B_DATABASE_WRITE_PATH=PASS`. This closes Node-RED B's MQTT and database dependency layer. Keep Node-RED B stopped; the next boundary is standby runtime bundle staging, which must use an approved pinned Node-RED image/digest rather than an invented tag.

Node-RED immutable image pin PASS - 2026-08-28

The official Node-RED runtime candidate was pulled on `u1c-03` and resolved to immutable image `nodered/node-red@sha256:10f40d0a83e7e5852b13d4d472b2006b05b1cca6d55e2f29a55a12c25a630cb6`. Runtime inspection reported Node-RED v5.0.4, Node.js v24.18.1, and npm 11.19.0. No Node-RED container was left running (`NODE_RED_RUNNING=NO`). Use this exact digest for both Node-RED A and Node-RED B so failover does not change application runtime versions. Do not substitute a mutable tag during deployment.

Node-RED pinned-image runtime identity PASS - 2026-08-28

Pinned image `nodered/node-red@sha256:10f40d0a83e7e5852b13d4d472b2006b05b1cca6d55e2f29a55a12c25a630cb6` was inspected with a temporary shell only; Node-RED itself was not started. Image config user is `node-red`; effective runtime identity is `uid=1000(node-red) gid=1000(node-red)` with no supplementary groups beyond GID 1000. `/data` in the image is `1000:0 mode 775`. Use UID/GID 1000 as the filesystem ownership boundary for the host-mounted Node-RED data directory and for read access to each candidate's own MQTT client key. Do not give B access to A's key, and keep the standby container stopped during staging.

Node-RED B filesystem preflight PASS - 2026-08-28

On `u1c-02`, `opsadmin` is UID/GID `1001:1001` while host account `jervis` owns numeric UID/GID `1000:1000`; the pinned Node-RED image also runs as numeric `1000:1000`. The dedicated host group `node-red-secrets` is absent, and `/etc/lorawan-cloud/node-red`, `/etc/lorawan-pki/node-red-mqtt`, and `/srv/node-red/data` are all absent. The pinned Node-RED image is not yet local on `u1c-02`, and no Node-RED container is running. Therefore do not make the MQTT client key readable by host group `1000`; create a dedicated `node-red-secrets` group and add that group to the container with `Compose group_add`. Use numeric owner `1000:1000` only for the bind-mounted `/srv/node-red/data` directory required by the container runtime. Keep Node-RED B stopped while staging.

Node-RED B filesystem foundation PASS - 2026-08-28

On `u1c-02`, system group `node-red-secrets` was created as GID 987. `/etc/lorawan-cloud/node-red` is `root:root mode 0750`; `/etc/lorawan-pki/node-red-mqtt` is `root:node-red-secrets mode 0750`; `/srv/node-red/data` is `numeric 1000:1000 mode 0700`. `opsadmin` is not a member of `node-red-secrets`, and no Node-RED container is running. This closes the standby filesystem foundation without transferring any private key or starting the standby.

Node-RED B image and secret-group staging PASS - 2026-08-28

On `u1c-02`, the exact pinned image `nodered/node-red@sha256:10f40d0a83e7e5852b13d4d472b2006b05b1cca6d55e2f29a55a12c25a630cb6` was pulled successfully and its RepoDigest matched exactly. A temporary shell from that image with supplementary group GID 987 showed effective identity `uid=1000(node-red) gid=1000(node-red) groups=987,1000`, proving the `Compose group_add` design can grant read access to `root:node-red-secrets` MQTT key material without adding the host login user to

that group. Node-RED B remained stopped. Proceed with host-specific standby certificate/key installation only; do not copy Node-RED A's key.

Node-RED B transfer SSH identity access-path stop - 2026-08-28

The first read-only inspection of possible standby certificate-transfer staging from ulc-03 to ulc-02 used the default opsadmin SSH client and was rejected with Permission denied (publickey). The local child-shell wrapper returned REMOTE_INSPECTION_RC=255 while ULC03_LOGIN_SHELL_SURVIVED=YES, so this is an SSH identity-selection stop, not a server outage or certificate-transfer failure. Existing project evidence already proves the root-controlled deployment identity /root/.ssh/cloud-deployment-phase8 on ulc-03 can authenticate as opsadmin to 10.104.0.4 when invoked from a root shell with IdentitiesOnly=yes and StrictHostKeyChecking=yes. Do not create a new SSH key, enable password SSH, or weaken host-key checking. Re-run only the read-only staging inspection through that existing deployment key before copying any certificate or private-key material.

Node-RED B transfer path PASS - 2026-08-28

The protected ulc-03 -> ulc-02 operator path was re-proven using the existing root-controlled deployment key /root/.ssh/cloud-deployment-phase8 with IdentitiesOnly=yes, StrictHostKeyChecking=yes, and BatchMode=yes. The remote identity resolved to opsadmin@ulc-02; no prior .node-red-b-transfer-* staging directory existed, so the earlier unauthenticated attempt transferred nothing. NODE_RED_B_TRANSFER_INSPECTION=PASS, TRANSFER_INSPECTION_RC=0, and ULC03_LOGIN_SHELL_SURVIVED=YES are authoritative. Proceed with one fresh protected transfer of only the already-issued node-red-ingest-standby certificate/private key; keep Node-RED B stopped.

Node-RED B transfer V2 heredoc-consumption harness stop - 2026-08-28

The V2 standby MQTT identity transfer re-verified the authoritative node-red-ingest-standby source certificate and key on ulc-03, including certificate SHA-256

f3d251a6abce3e20bc99bfe456fb5f7c3cf071acded4553bdc6a76d42fd82fbd and public-key SHA-256

0d0a041770d56883e4c335ed4691cc2ae8c279f03e672ca4f5f6e0755888d4f1, then stopped immediately during the first remote staging-directory SSH command. The wrapper was running inside sudo bash <<'EOF', but its SSH array omitted -n; SSH therefore inherited the wrapper heredoc on stdin and could consume the remaining local script. The enclosing child shell then reached EOF and returned 0, so the printed

NODE_RED_B_MQTT_IDENTITY_TRANSFER_EXIT=0 does not mean steps 4-7 executed. No scp section output appeared, so do not claim certificate/key transfer PASS. Treat this as a harness stop after source verification. Before retrying, inspect ulc-02 read-only for any .node-red-b-transfer-* directory using the proven /root/.ssh/cloud-deployment-phase8 identity with ssh -n; remove nothing until the observed staging state is known. The ulc-03 login shell survived.

Node-RED B V2 staging inspection PASS - 2026-08-28

The corrected read-only inspection on ulc-03 used the existing root-controlled deployment key and ssh -n to inspect ulc-02. It found exactly one V2 staging directory, /home/opsadmin/.node-red-b-transfer-20260828T065003Z, owned opsadmin:opsadmin mode 0700, with both client.crt and client.key absent. This proves the prior V2 wrapper stopped after remote directory creation because SSH consumed the remaining local heredoc; no certificate or private key crossed hosts. The next safe action is to remove only that empty staging directory, create a fresh protected staging directory, and perform the transfer with every remote SSH invocation using -n.

Node-RED B MQTT identity transfer V3 partial PASS - 2026-08-28

The corrected V3 transfer from ulc-03 to ulc-02 used the existing root-controlled deployment SSH identity and successfully copied only the standby files into /home/opsadmin/.node-red-b-transfer-20260828T065353Z. The destination certificate SHA-256 is f3d251a6abce3e20bc99bfe456fb5f7c3cf071acded4553bdc6a76d42fd82fbd;

certificate and private-key public-key SHA-256 both equal

0d0a041770d56883e4c335ed4691cc2ae8c279f03e672ca4f5f6e0755888d4f1; subject is CN = node-red-ingest-standby; fingerprint is

2B:C4:A2:7A:42:57:E4:E4:15:BC:CA:0E:1E:8C:17:7D:1D:EA:F5:21:97:43:3F:D5:3A:ED:C3:DB:F9:D0:DF:AC; staging is 0700 opsadmin:opsadmin with both files 0600 opsadmin:opsadmin; and Node-RED B remained stopped. The destination openssl verify against /etc/lorawan-pki/mqtt/ca.crt did **not** run successfully because unprivileged opsadmin could not traverse/read that protected CA path (Permission denied). The remote command did not use set -e, so its later DESTINATION_IDENTITY=PASS line is not authoritative for chain verification. Treat transfer integrity/key matching as PASS, but complete the CA-chain verification locally on ulc-02 under sudo before installing the files into /etc/lorawan-pki/node-red-mqtt.

Node-RED B MQTT identity final install PASS - 2026-08-28

On ulc-02, the staged node-red-ingest-standby certificate was verified against /etc/lorawan-pki/mqtt/ca.crt for sslclient under sudo, then installed with its matching private key under /etc/lorawan-pki/node-red-mqtt/. Both final files are root:node-red-secrets mode 0640 (numeric group GID 987). The certificate chain and certificate/private-key public-key match both passed. A temporary shell from the exact pinned Node-RED image with supplementary GID 987 proved the runtime can read the MQTT CA, client certificate, and client key. The protected transfer staging directory was deleted and no Node-RED container is running.

NODE_RED_B_MQTT_IDENTITY_INSTALL=PASS; the standby MQTT identity staging boundary is complete.

Node-RED PostgreSQL palette pin PASS - 2026-08-28

Pinned palette node-red-contrib-postgresql@0.16.2 was discovered from the already-pinned Node-RED image without starting Node-RED. npm metadata reported engines.node >=8, published integrity sha512-aPZBngjBgShw4YvK/c0FKPYyvgM2/1XFSBbTbjNN6kUc6aqIxIRGvyFwtgWRw50nfcPKj/zq6y0Nc/HOTuyWAA==, and the package README explicitly documents parameterized queries through msg.params plus named parameters through msg.queryParameters. NODE_RED_POSTGRES_PALETTE_DISCOVERY=PASS; no Node-RED container was left running. Use exact version 0.16.2 in the shared A/B package manifest and lock file.

Node-RED shared runtime dependency lock PASS - 2026-08-28

The shared runtime dependency artifact is now repository-backed under deployment/server/integrations/node-red/runtime/. package.json pins node-red-contrib-postgresql exactly to 0.16.2; the npm-generated lock file is lockfileVersion 3 and has SHA-256 89289e301cab799ac7e85e2fbe2fc40b34ff195e799313a4f720c642397ba85e. The locked package entry retains the published integrity sha512-

aPZBngjBgShw4YvK/c0FKPYyvgM2/1XFSBbTbjNN6kUc6aqIxIRGvyFwtgWRw50nfcPKj/zq6y0Nc/HOTuyWAA==. The same runtime directory also contains a shared secret-free compose.yml, guarded settings.js, and node-red.env.example; neither Node-RED candidate was started by this repository work. Host-specific IP/GID/MQTT client identity and protected credential values remain outside the shared bundle.

Node-RED shared-secret bootstrap verifier stop - 2026-08-28

The first shared-secret bootstrap stopped during the credential-secret size verifier, not during proven secret generation. The command used sudo wc -c < /root/node-red-ha-bootstrap/credential-secret; the < redirection was performed by the non-root login shell before sudo, so Bash returned Permission denied and the numeric test then received an empty value. Do not regenerate the credential secret until a root-only metadata check determines whether /root/node-red-ha-bootstrap/credential-secret already exists with the expected protected metadata/size. Keep the login shell safe by running strict mode only in a child shell.

Node-RED shared credential secret state PASS - 2026-08-28

On ulc-03, /root/node-red-ha-bootstrap is 0700 root:root. The already-generated credential-secret is 0600 root:root and exactly 65 bytes (64 hex characters plus newline), so the shared Node-RED credential-encryption

secret is valid and must not be regenerated. `admin-password.hash` is not yet present, and no Node-RED container is running. The next bounded step is to generate only the editor bcrypt hash using the pinned Node-RED image and store it root-only without printing or committing it.

Node-RED admin bcrypt non-interactive harness stop - 2026-08-28

The first admin-password bootstrap retry preserved the existing shared credential secret but did not create `admin-password.hash`. The wrapper piped the password into `node-red admin hash-pw` and attempted to capture stdout, but the Node-RED CLI password-hash helper is an interactive terminal workflow; no bcrypt hash was emitted into the captured stream, so the harness stopped at `BCRYPT_HASH_GENERATION=FAIL`. No Node-RED service was started and no admin hash file was written. Retry with a real pseudo-terminal and a root-only temporary transcript, extract only the bcrypt output, install it `0600 root:root`, then remove the transcript.

Node-RED admin bcrypt bootstrap PASS - 2026-08-28

The Node-RED editor bcrypt hash was generated interactively with the pinned Node-RED image, extracted successfully, stored only at `/root/node-red-ha-bootstrap/admin-password.hash` as `root:root mode 0600` with expected size 61 bytes, and the temporary transcript was removed. The existing shared `credential-secret` was preserved. `NODE_RED_RUNNING=NO` remained true. Do not record or copy the bcrypt value into repository documentation.

Node-RED telemetry_writer host-psql harness stop - 2026-08-28

The first attempt to stage the existing `telemetry_writer` password on `ulc-03` stopped before any database authentication or secret-file write because the host `psql` command is Ubuntu `pg_wrapper` without an installed versioned `postgresql-client-<version>` package. The wrapper printed You must install at least one `postgresql-client-<version>` package and exited before connecting. This is a local client-tooling harness limitation, not a PgBouncer, PostgreSQL, SCRAM, or password failure. Do not install a host PostgreSQL client solely for this step. Reuse the already-proven Phase 9 pattern: run `psql` from a disposable container based on the current `spilo` image, mount the commissioned PgBouncer CA read-only, connect to logical host `pgbouncer.internal.lorawan.com` with physical `hostaddr=10.104.0.8`, and store the root-only Node-RED copy only after the real login succeeds.

Node-RED telemetry_writer SASL authentication stop - 2026-08-28

The disposable Spilo PostgreSQL client reached `ulc-03` PgBouncer at `10.104.0.8:6432` with logical host `pgbouncer.internal.lorawan.com`, but PgBouncer returned `FATAL: SASL authentication failed for telemetry_writer`. This is real authentication-path evidence, unlike the preceding `host-psql` tooling stop. No Node-RED secret file was written because the wrapper stopped before the storage step. Do not rotate the PostgreSQL role yet. First compare the sanitized hash of the `telemetry_writer` SCRAM verifier in the commissioned PgBouncer userlists on `ulc-03` and `ulc-02`; if they match, retry the hidden operator password once to rule out an input typo. If they differ, repair PgBouncer verifier parity before any password rotation.

Node-RED telemetry_writer confirmed-entry SASL stop - 2026-08-28

The operator re-entered the existing `telemetry_writer` password twice and the two hidden entries matched (`PASSWORD_CONFIRMATION=PASS`), but a disposable Spilo PostgreSQL client again reached `ulc-03` PgBouncer at `10.104.0.8:6432` and returned `FATAL: SASL authentication failed`. This rules out a simple mistyped confirmation within that retry. No Node-RED secret file was created. Stop password guessing. Before any credential rotation, compare the protected `telemetry_writer` SCRAM verifier in `/etc/pgbouncer/userlist.txt` on `ulc-03` with the authoritative current PostgreSQL verifier from `pg_authid`, using only one-way hashes or equality output and never printing either verifier. If they differ, repair PgBouncer verifier parity. If they match, the entered plaintext password is not the current commissioned role password and must be recovered from approved secret custody or deliberately rotated through a controlled multi-node PgBouncer update.

Node-RED telemetry_writer verifier parity PASS - 2026-08-28

The confirmed credential retry reached ulc-03 PgBouncer but failed SCRAM twice with matching hidden operator entries. A protected read-only comparison then showed the telemetry_writer SCRAM verifier in /etc/pgbouncer/userlist.txt is byte-equivalent to PostgreSQL's current authoritative pg_authid verifier (PGBOUNCER_VS_POSTGRES_VERIFIER_PARITY=PASS). Therefore PgBouncer is not stale and the entered plaintext is not the current commissioned password. Repository review found no approved plaintext custody file for telemetry_writer; do not scrape or reverse the SCRAM verifier. Because Node-RED has not been started yet, rotate telemetry_writer once through the controlled PostgreSQL + three-node PgBouncer workflow, then stage the new password root-only for Node-RED.

Node-RED telemetry_writer rotation secret preparation PASS - 2026-08-28

On ulc-03, a replacement telemetry_writer password was generated as 64 lowercase hexadecimal characters plus newline and stored only at /root/node-red-ha-bootstrap/telemetry-writer-password.pending with 0600 root:root protection. No active Node-RED secret file exists yet, and no Node-RED container is running. This is preparation only; PostgreSQL and PgBouncer have not yet been changed. The next mutation is a controlled role-password rotation against the actual Patroni leader, followed by verifier refresh on PgBouncer one node at a time.

Node-RED telemetry_writer rotation admin-path harness stop - 2026-08-28

The rotation preflight correctly detected exactly one Patroni leader at 10.104.0.2 and the existing Spilo superuser path successfully connected to that leader over verify-full TLS as postgres. SQL returned postgres|10.104.0.2|false, proving the session reached the writable primary. The wrapper then reported POSTGRES_SUPERUSER_ADMIN_PATH=FAIL only because its case expression expected the abbreviated boolean text f; PostgreSQL boolean concatenation rendered the full text false. Treat this as a comparison-harness defect only. No PostgreSQL role, PgBouncer userlist, Node-RED secret, or service state was changed. The admin path itself is authoritative PASS and does not need to be rerun before the controlled telemetry_writer password rotation.

Node-RED telemetry_writer PostgreSQL rotation PASS - 2026-08-28

The controlled telemetry_writer password rotation completed on the single current Patroni leader at 10.104.0.2. The pending replacement secret remained root-only at /root/node-red-ha-bootstrap/telemetry-writer-password.pending. PostgreSQL returned ALTER ROLE, generated a different valid SCRAM-SHA-256 verifier, and a fresh direct sslmode=verify-full login using the replacement password returned telemetry_writer|lorawan_telemetry|false. Node-RED remained stopped. At this checkpoint PostgreSQL holds the new verifier while PgBouncer userlists still hold the old verifier; that temporary mismatch is intentional. Refresh PgBouncer one node at a time, verify a new connection after each refresh, and do not promote the pending Node-RED secret to its final path until the intended PgBouncer endpoints accept it.

Node-RED ulc-03 PgBouncer telemetry_writer refresh PASS - 2026-08-28

The controlled credential rotation advanced through ulc-03 PgBouncer successfully. The existing four-entry /etc/pgbouncer/userlist.txt was confirmed to contain the old telemetry_writer verifier, a fresh four-role candidate was regenerated from authoritative PostgreSQL SCRAM verifiers, and the new telemetry_writer entry matched PostgreSQL. The old file was preserved at /etc/pgbouncer/userlist.txt.before-telemetry-writer-20260828T075208Z; the new file remained 0640 root:postgres. PgBouncer was reloaded in-place with PID 789143 unchanged, and the pending rotated password authenticated through local pgbouncer.internal.lorawan.com / 10.104.0.8:6432 to lorawan_telemetry, returning pg_is_in_recovery() = false. ULC03_PGBOUNCER_TELEMETRY_WRITER_REFRESH=PASS. Node-RED remained stopped and the new password remains pending root-only until all PgBouncer nodes accept it. Next refresh ulc-02 only, verify its local :6432, then continue to the remaining node.

Node-RED ulc-02 PgBouncer telemetry_writer refresh PASS - 2026-08-28

The controlled telemetry_writer rotation advanced through ulc-02 PgBouncer successfully. On host ulc-02, the existing four-entry /etc/pgbouncer/userlist.txt was confirmed stale relative to the replicated PostgreSQL verifier, a fresh four-role SCRAM userlist was regenerated from the local Spilo/PostgreSQL member, and the candidate telemetry_writer verifier matched PostgreSQL. The prior userlist was preserved at /etc/pgbouncer/userlist.txt.before-telemetry-writer-20260828T075607Z; the installed file remains 0640 root:postgres. PgBouncer reloaded in place with PID 787012 unchanged, final local verifier parity passed, and Node-RED B remained stopped. ULC02_PGBOUNCER_TELEMETRY_WRITER_REFRESH=PASS is authoritative. Next, from ulc-03, use the root-only pending password to perform one verify-full authentication through physical endpoint 10.104.0.4:6432; only after that passes should the remaining ulc-01 PgBouncer verifier be refreshed.

Node-RED ulc-02 rotated credential end-to-end PASS - 2026-08-28

The pending rotated telemetry_writer password was then tested from ulc-03 through the standby database endpoint pgbouncer.internal.lorawan.com with physical hostaddr=10.104.0.4, port 6432, and sslmode=verify-full. Authentication succeeded as telemetry_writer to lorawan_telemetry and returned pg_is_in_recovery() = false, proving the full ulc-02 PgBouncer -> local HAProxy :15432 -> writable Patroni primary path accepts the new credential. ULC02_ROTATED_CREDENTIAL_END_TO_END=PASS. Node-RED A remained stopped. The next bounded mutation is to refresh only ulc-01 PgBouncer from its local authoritative PostgreSQL SCRAM verifier, reload in place, and verify the new credential before promoting the pending Node-RED secret.

Node-RED ulc-01 PgBouncer telemetry_writer refresh PASS - 2026-08-28

The final PgBouncer node was refreshed successfully on ulc-01. The existing four-entry /etc/pgbouncer/userlist.txt was confirmed stale only for the rotated telemetry_writer verifier, a fresh four-role userlist was regenerated from authoritative PostgreSQL SCRAM verifiers, and the candidate telemetry_writer entry matched PostgreSQL. The previous file was preserved at /etc/pgbouncer/userlist.txt.before-telemetry-writer-20260828T075958Z; the active file remains 0640 root:postgres. PgBouncer reloaded in place with PID 1105819 unchanged, and final local verifier parity passed. ULC01_PGBOUNCER_TELEMETRY_WRITER_REFRESH=PASS. All three PgBouncer nodes now carry the rotated telemetry_writer verifier. The remaining checkpoint is one final verify-full login through each :6432 endpoint using the protected pending password, then promote /root/node-red-ha-bootstrap/telemetry-writer-password.pending to the active Node-RED secret only if all three paths pass.

Node-RED telemetry_writer three-node rotation finalization PASS - 2026-08-28

The controlled telemetry_writer rotation is complete. The protected replacement credential authenticated through all three commissioned PgBouncer endpoints (10.104.0.2:6432, 10.104.0.4:6432, and 10.104.0.8:6432) using logical hostname pgbouncer.internal.lorawan.com and verify-full TLS; every session reached lorawan_telemetry as telemetry_writer and returned pg_is_in_recovery() = false. THREE_NODE_PGBOUNCER_AUTH=PASS is authoritative. Only after all three paths passed, /root/node-red-ha-bootstrap/telemetry-writer-password.pending was promoted to /root/node-red-ha-bootstrap/telemetry-writer-password, which remains 0600 root:root and 65 bytes (64 hexadecimal characters plus newline). Node-RED remained stopped. The PostgreSQL/PgBouncer credential-recovery problem is closed; do not repeat these authentication or verifier-rotation gates unless a later runtime failure provides a reason. The next boundary is protected node-red.env staging for Node-RED B on ulc-02, using the shared credential secret, shared editor bcrypt hash, rotated telemetry_writer password, host-local NODE_RED_SECRET_GID=987, NODE_RED_LOCAL_IP=10.104.0.4, and MQTT client ID node-red-ingest-standby.

Node-RED B protected env staging group-lookup harness stop - 2026-08-28

The first protected node-red.env staging attempt transferred the root-built environment and installer to ulc-02 successfully, then stopped before installing /etc/lorawan-cloud/node-red/node-red.env. The remote verifier incorrectly used id -g node-red-secrets, which asks for a user named node-red-secrets; the commissioned

boundary is a system group, previously created as GID 987. That command therefore returned `id: node-red-secrets: no such user`, leaving an empty numeric comparison and aborting under `set -e`. Treat this as a verifier-harness defect only: the protected target env was not installed, Node-RED B was not started, and the transfer wrapper cleanup path removed the temporary staging payload. Correct the retry by resolving the group through `getent group node-red-secrets` (or equivalent group database lookup), require GID 987, then perform the same root-only install and secret-free structural validation without printing secret values.

Node-RED B protected env install PASS with remote-shell cleanup pending - 2026-08-28

The corrected V2 environment staging reached and completed the protected install on `ulc-02`: the `node-red-secrets` group resolved to GID 987, transfer SHA parity passed, `/etc/lorawan-cloud/node-red/node-red.env` was installed `root:root mode 0600` with size 551, all expected non-secret fields and secret formats validated, and Node-RED B remained stopped. However, the operator transcript ended at an interactive `root@ulc-02` prompt after `NODE_RED_B_PROTECTED_ENV_STAGING=PASS`, so the remote shell had not yet exited and its `EXIT` cleanup trap had not yet been observed. Treat the environment install itself as `PASS`, but do not claim temporary transfer-directory cleanup or outer-wrapper completion until the operator exits that one remote root shell and the `ulc-03` wrapper returns.

Node-RED B protected env wrapper interrupt after install PASS - 2026-08-28

The V2 protected environment retry successfully verified the commissioned `node-red-secrets` group as GID 987, transfer integrity, installed `/etc/lorawan-cloud/node-red/node-red.env` as `0600 root:root`, validated the expected host-specific and protected values without printing secrets, and confirmed Node-RED B was not running. The operator then interrupted the still-open remote interactive root shell with `^C`; the outer `ulc-03` wrapper consequently reported `NODE_RED_B_REMOTE_INSTALL=FAIL / exit 1`. Treat this as a wrapper return-path/cleanup interruption only, not an environment-install failure. Do not reinstall the environment. Next inspect `ulc-02` read-only from `ulc-03` with `ssh -n` to confirm the target file metadata, Node-RED stopped state, and whether any `.node-red-env-transfer.*` staging directory remains. Remove only an observed leftover staging directory in a separate bounded cleanup step.

Node-RED B post-interrupt env inspection PASS - 2026-08-28

The interrupted protected-environment wrapper was inspected from `ulc-03` without reinstalling anything. `/etc/lorawan-cloud/node-red/node-red.env` still exists on `ulc-02` as `root:root mode 0600`, size 551, and Node-RED B remains stopped. Exactly one temporary transfer directory remains: `/home/opsadmin/.node-red-env-transfer.GtVASVo0`. Treat the environment install itself as `PASS`. The only remaining cleanup is to remove that exact staging directory, then recheck that no `.node-red-env-transfer.*` directories remain and that the protected env and stopped-standby state are unchanged.

Node-RED B protected env staging and cleanup PASS - 2026-08-28

The protected standby environment staging is complete. Post-interrupt inspection proved `/etc/lorawan-cloud/node-red/node-red.env` persisted as `root:root mode 0600` with size 551, while Node-RED B remained stopped. The sole leftover transfer directory `/home/opsadmin/.node-red-env-transfer.GtVASVo0` was then removed explicitly; a follow-up count returned zero matching staging directories, the protected environment file remained unchanged, and `NODE_RED_B_TRANSFER_CLEANUP=PASS` completed with exit code 0. Do not reinstall or recreate the protected environment. The next boundary is the secret-free shared runtime bundle (`compose.yml`, `settings.js`, `package.json`, `package-lock.json`) followed by `docker compose --env-file node-red.env config --quiet`; keep Node-RED B stopped and continue to defer `flows.json` until its PostgreSQL node configuration is runtime-validated.

Node-RED B shared runtime bundle preparation PASS - 2026-08-28

The secret-free Node-RED shared runtime bundle was prepared and independently verified on `u1c-03` before any standby-host mutation. The archive `/tmp/node-red-runtime-bundle.tgz` has SHA-256

`9f2a8b708e6818fc504120aef9acc13801e565e9912afb22d33935cee4de8afb` and contains exactly `compose.yml`, `settings.js`, `package.json`, and `package-lock.json`. Extracted file SHA-256 values matched the repository artifacts: `compose.yml` `979b6e4f468bec0af375801c3e98d03f127832daee489b0f73b61e412b2154c8`, `settings.js` `eeb17fb546ea87173a2359adc24b87db0016a388cfc79f6233b82e3e4469b55a`, `package.json` `ea92c1cdabb1779565027b35e8b4725e7d75d53f54c10e40231ff1d723dacb81`, and `package-lock.json` `89289e301cab799ac7e85e2fbc40b34ff195e799313a4f720c642397ba85e`.

`NODE_RED_RUNTIME_BUNDLE_PREPARATION=PASS`; Node-RED B remained stopped. The next mutation is to transfer this verified archive to `u1c-02`, install `compose.yml` under `/etc/lorawan-cloud/node-red`, install `settings.js`, `package.json`, and `package-lock.json` under the bind-mounted `/srv/node-red/data` user directory with numeric ownership `1000:1000`, then run only `docker compose --env-file node-red.env config --quiet`. Do not start Node-RED B and do not create `flows.json` in this step.

Node-RED B shared runtime staging PASS - 2026-08-28

The standby shared runtime layer was installed and validated successfully on `u1c-02` from the previously verified secret-free archive. The transferred archive and all four component hashes matched the repository-reviewed sources. `/etc/lorawan-cloud/node-red/compose.yml` was installed, while `settings.js`, `package.json`, and `package-lock.json` were installed under `/srv/node-red/data` with numeric ownership `1000:1000`. The pinned-image node `--check /data/settings.js` syntax validation passed, and `docker compose --env-file node-red.env config --quiet` parsed the deployment definition successfully without creating or starting Node-RED. `flows.json` remains intentionally absent and Node-RED B remains stopped. `NODE_RED_B_SHARED_RUNTIME_STAGING=PASS` and `NODE_RED_B_RUNTIME_STAGING_EXIT=0` are authoritative. The next boundary is to generate and runtime-validate the reviewed telemetry `flows.json` using the exact `node-red-contrib-postgresql@0.16.2` node/configuration contract; do not guess palette node fields.

Node-RED PostgreSQL palette exact-contract BusyBox grep harness stop - 2026-08-28

The exact `node-red-contrib-postgresql@0.16.2` package download and integrity verification passed, and package metadata confirmed Node-RED registration `postgresql -> postgresql.js`. The intended field/query/TLS inspection then did not execute because the pinned Node-RED image provides BusyBox `grep`, which does not support `GNU --include`; each `grep` printed its usage text and the wrapper continued because those commands were deliberately suffixed with `|| true`. Therefore `POSTGRESQL_NODE_CONTRACT_INSPECTION=PASS` is not authoritative for sections 4-6 and no `flows.json` fields may be inferred from that run. The extracted package files in `/tmp/node-red-postgresql-inspect.H2A35T` were created as root by the disposable container, so the normal-user cleanup trap could not remove them. Reuse that exact already-verified package for a direct-file inspection using BusyBox-compatible commands, then remove only that temp directory with `sudo` after capture. Node-RED remained unchanged/stopped.

Node-RED PostgreSQL palette exact-contract capture PASS - 2026-08-28

The exact `node-red-contrib-postgresql@0.16.2` package contract was captured from the previously verified tarball on `u1c-03`. The package registers config node type `postgresqlConfig` and execution node type `postgresql`. Runtime code confirms `msg.query` overrides the editor query, positional bind values come directly from `msg.params`, and named parameters may alternatively arrive through `msg.queryParameters`. The config node supports environment-backed database identity through `userFieldType=env / userEnv` and `passwordFieldType=env / passwordEnv`, allowing the shared flow to reference `TELEMETRY_DB_USER` and `TELEMETRY_DB_PASSWORD` without plaintext database credentials in `flows.json`. The config node passes the resolved `ssl` value directly to `pg.Pool`; with the commissioned logical hostname and `NODE_EXTRA_CA_CERTS=/run/pgbouncer/ca.crt`, the cloud flow should keep `host=pgbouncer.internal.lorawan.com`, `port=6432`, and TLS enabled. The package-supplied example also confirms the serialized config-node and execution-node field names. Temporary inspection files were removed

successfully and no Node-RED container was running. `POSTGRESQL_NODE_CONTRACT_CAPTURE=PASS` is authoritative. The remaining schema dependency before generating `flows.json` is the exact built-in Node-RED MQTT broker/TLS node serialization from the pinned Node-RED 5.0.4 image.

Node-RED shared flows.json generation PASS - 2026-08-28

The exact Node-RED 5.0.4 MQTT/TLS node schema and `node-red-contrib-postgresql` 0.16.2 schema were captured from the pinned runtime. The repository now contains `deployment/server/integrations/node-red/runtime/flows.json` with the shared application-uplink subscription `application/+device/+event/up` at QoS 0, mTLS through `mqtt.internal.lorawan.com:18884`, host-specific client ID via `${NODE_RED_MQTT_CLIENT_ID}`, env-backed `telemetry_writer` credentials, verified TLS to `pgbouncer.internal.lorawan.com:6432`, and the reviewed EMU-01 payload-v2 normalization/parameterized SQL function. The SQL idempotency gate was tightened with `WHERE uplink_state.count > 0` so duplicate uplink conflicts cannot create later measurement rows from a changed mapping. No private key, database password, MQTT password, or populated credential value is stored in the flow. Node-RED remained stopped; dependency installation and runtime loading are the next validation boundary.

Node-RED B palette and flows staging PASS - 2026-08-28

On `ulc-02`, the locked `node-red-contrib-postgresql@0.16.2` palette was installed with `npm ci` from the repository `package-lock.json`, the installed package version verified as `0.16.2`, and the repository-backed `flows.json` was installed under `/srv/node-red/data` with the expected SHA-256 `02be61d7fafdaa8877b9b6f5cf5ef32f7685730e300d4af55b49aadd76518718`. Static flow validation passed, including unique node IDs, Function-node syntax, the MQTT application-uplink subscription contract, environment-backed PostgreSQL credentials, and no `flows_cred.json`. Node-RED B remained stopped. This closes the standby runtime staging boundary; the next step is to bring Node-RED A on `ulc-03` to the same reviewed runtime revision before starting either instance.

Node-RED A current-state preflight PASS - 2026-08-28

On `ulc-03`, Node-RED A is not running and the exact pinned Node-RED image is already local. The dedicated `node-red-secrets` group is absent; `/etc/lorawan-cloud/node-red`, `/etc/lorawan-pki/node-red-mqtt`, and `/srv/node-red/data` are all absent; no protected env, Compose file, runtime files, `flows.json`, or `flows_cred.json` exist yet. The authoritative original MQTT workload issuance directory `/root/lorawan-pg-ca/node-red-ingest-issuance-20260827T142128Z` is present with `client.crt` SHA-256 `eb5cc28f5eb89c1586d8aae387b52edfa09e1918e6aa74fae2571d81f4e7e576` and subject `CN = node-red-ingest`. This is a clean A-side bootstrap boundary. Create the dedicated secret group and host directories first, verify their ownership/modes and keep A stopped, then install the already-issued `node-red-ingest` certificate/key in a separate bounded step.

Node-RED A filesystem foundation PASS - 2026-08-28

On `ulc-03`, the Node-RED A filesystem/security foundation is now complete. The dedicated system group `node-red-secrets` exists as GID 987; `/etc/lorawan-cloud/node-red` is `root:root mode 0750`; `/etc/lorawan-pki/node-red-mqtt` is `root:node-red-secrets mode 0750`; and `/srv/node-red/data` is `numeric 1000:1000 mode 0700`. The exact pinned Node-RED image was already local and Node-RED A remained stopped throughout. No MQTT certificate/key, protected env, compose file, palette, or flows were installed by this step. The next bounded mutation is to install only the already-issued `node-red-ingest` client certificate/private key from the authoritative `ulc-03` CA issuance directory, verify `clientAuth` chain and `cert/key` match, prove the pinned container can read the protected files using supplementary GID 987, and keep Node-RED A stopped.

Node-RED A MQTT identity install PASS - 2026-08-28

On `ulc-03`, the already-issued `node-red-ingest` MQTT workload identity was installed under `/etc/lorawan-pki/node-red-mqtt/`. The certificate and private key are `root:node-red-secrets mode 0640`; the certificate SHA-

256 remained eb5cc28f5eb89c1586d8aae387b52edfa09e1918e6aa74fae2571d81f4e7e576, the certificate/private-key public-key SHA-256 matched at 51e1578332155e45fb692a4b8e834b2c0545b22a0108b7812dd8228fcd9920d9, and openssl verify -purpose sslclient passed against the commissioned MQTT CA. A disposable shell from the exact pinned Node-RED image with supplementary group GID 987 proved runtime read access to the CA, client certificate, and private key. Node-RED A remained stopped. NODE_RED_A_MQTT_IDENTITY_INSTALL=PASS is authoritative. Next stage the protected A node-red.env from the existing root-only shared bootstrap secrets using A-specific local IP 10.104.0.8, MQTT client ID node-red-ingest, and secret-group GID 987; do not print any secret values.

Node-RED A protected environment staging PASS - 2026-08-29

On ulc-03, /etc/lorawan-cloud/node-red/node-red.env is installed as root:root mode 0600 with size 543 bytes. The three existing root-only bootstrap secrets were format-checked and the installed environment was validated without printing any secret value. Host-specific values are NODE_RED_SECRET_GID=987, NODE_RED_LOCAL_IP=10.104.0.8, NODE_RED_MQTT_CLIENT_ID=node-red-ingest, region as923, and database role telemetry_writer. NODE_RED_A_PROTECTED_ENV_STAGING=PASS and NODE_RED_A_ENV_STAGING_EXIT=0 are authoritative. Node-RED A remained stopped. The canonical LF-normalized shared runtime files were rechecked against the exact hashes already staged on Node-RED B and all five files (compose.yml, settings.js, package.json, package-lock.json, flows.json) matched byte-for-byte. Next stage those canonical bytes on A, install the locked PostgreSQL palette, validate Compose and flow syntax, and keep A stopped until controlled activation.

Node-RED A complete runtime staging PASS - 2026-08-29

On ulc-03, Node-RED A now carries the exact same reviewed shared runtime revision as Node-RED B. The canonical LF-normalized compose.yml, settings.js, package.json, package-lock.json, and flows.json hashes matched the already-staged standby, including flows.json SHA-256 02be61d7fafdaa8877b9b6f5cf5ef32f7685730e300d4af55b49aadd76518718. The locked node-red-contrib-postgresql palette installed as version 0.16.2; settings syntax, static flow validation, and docker compose --env-file node-red.env config --quiet all passed. Node-RED A remained stopped throughout. Both A and B are now fully staged at the same application revision. The next boundary is controlled activation of A only, after re-proving B is fenced/stopped; then verify Node-RED startup, MQTT subscription, and database connectivity before sending one real uplink.

Atomic telemetry + outbox revision pre-mutation gate PASS - 2026-08-29

After later server-first review identified the intentionally deferred outbox enqueue as remaining application work, a read-only live gate re-proved the current boundary before any rollout. Node-RED A remained running|0|healthy; B remained fenced/stopped. Both still used the prior flow SHA-256 02be61d7fafdaa8877b9b6f5cf5ef32f7685730e300d4af55b49aadd76518718 and identical Compose SHA-256 5607fddf6a31eea71376d720c2f2f24903818635800a967fa276ca1f21f00934. Neither flow already contained telemetry.fabric_outbox, and FABRIC_SELECTED_DEV_EUI was absent from both protected environments.

The reserved synthetic identity 0000000000000000 had no existing uplink, measurement, or joined outbox rows. telemetry.fabric_outbox and its writer boundary passed: telemetry_writer has INSERT/SELECT plus sequence USAGE; status defaults to pending, attempts to 0, and next_attempt_at to now(). NODE_RED_ATOMIC_OUTBOX_PREMUTATION=PASS. No restart/configuration/file mutation occurred. The next journey step is standby-first: install only candidate compose.yml SHA-256 17aade702bf2206e9a4f2177fa8b0f47a7012da431a2adc7d4b064ce0b897730, candidate flows.json SHA-256 476056c5cff951ff46bb48c2eeb0e153b666c8cdc42eab88532fd3bebbcfdc753, and protected FABRIC_SELECTED_DEV_EUI=0000000000000000 on B; validate while B remains stopped; then advance to A only after B passes.

Node-RED B atomic-outbox standby staging COMPLETE / PASS - 2026-08-29

The new atomic telemetry + optional Fabric-outbox revision was staged on stopped Node-RED B first. The secret-free candidate was independently reconstructed and matched reviewed hashes

`compose.yml=17aade702bf2206e9a4f2177fa8b0f47a7012da431a2adc7d4b064ce0b897730` and

`flows.json=476056c5cff951ff46bb48c2eeb0e153b666c8cdc42eab88532fd3bebbcdc753`. B still matched the old hashes immediately before install. A remained running|0|healthy throughout.

A temporary rollback copy protected only the B mutation window. The candidate files were installed with existing ownership/modes preserved, protected `FABRIC_SELECTED_DEV_EUI=0000000000000000` was added exactly once, and the installed hashes matched the reviewed candidate. Flow JSON/atomic-outbox structural checks passed, `flows_cred.json` remained absent, and `docker compose --env-file node-red.env config --quiet` passed without starting the service. B remained fenced after installation; the temporary rollback copy was then removed. No synthetic event or database mutation occurred. `NODE_RED_B_ATOMIC_OUTBOX_STAGING_FINAL=PASS`. The next journey step is A-only rollout/recreate with B still fenced, followed by the isolated synthetic/replay proof only after A returns healthy.

Node-RED A/B atomic-outbox runtime COMPLETE / PASS - 2026-08-29

The reviewed atomic-outbox candidate was promoted to active A from the already verified stopped-B bytes. The candidate hashes were rechecked before install, protected `FABRIC_SELECTED_DEV_EUI=0000000000000000` was added exactly once, Compose validation passed, B was re-proven fenced, and A was recreated once. A returned running|0|healthy with restart count 0; editor exposure remained only 127.0.0.1:1880; `mqtt.internal.lorawan.com` and `pgbouncer.internal.lorawan.com` both resolved locally to 10.104.0.8; local dependency routes 18884 and 6432 passed; the runtime selector was present; no known PgBouncer-CA permission/TLS regression appeared; and final A/B candidate hash parity passed while B remained stopped.

The previously missing runtime-file-access line was closed with a read-only resume only. Runtime UID/GID is 1000:1000; all required MQTT/PgBouncer CA/client files and the four reviewed /data runtime files were readable.

The runtime PgBouncer and MQTT CA SHA-256 values both matched

`6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb`. The selector remained exactly once in A, B stayed fenced, candidate hashes stayed unchanged on both hosts, and A finished running|0|healthy.

`NODE_RED_RUNTIME_FILE_ACCESS=PASS`, `NODE_RED_RUNTIME_CA_ACCESS=PASS`,

`NODE_RED_A_ATOMIC_OUTBOX_ROLLOUT=PASS`, and `NODE_RED_A_B_ATOMIC_OUTBOX_RUNTIME=PASS` are authoritative. No further rollout/recreate is needed.

Synthetic atomic telemetry + Fabric outbox proof COMPLETE / PASS - 2026-08-29

Without editing the production flow or publishing a fake ChirpStack MQTT event, the test harness loaded the exact deployed `validate + normalize + parameterize` Function from active A and executed its generated SQL through the live PgBouncer path as the real `telemetry_writer` runtime. Reserved event `server-synthetic-NODERED-OUTBOX-SYNTH-20260829T152713Z` (DevEUI=0000000000000000, event time 2026-08-29T15:27:13.000Z) produced exactly 25 SQL parameters, thirteen normalized metrics, one `telemetry.uplinks` row, thirteen distinct measured `telemetry.measurements` rows, and one `telemetry.fabric_outbox` row. The outbox row is pending, attempts 0, unclaimed/unsealed, event type `lorawan_uplink_accepted`, schema `telemetry-attestation-v1`.

The exact event was then replayed. The final SQL insert returned rowcount 0 and counts remained 1|13|1; no duplicate telemetry or outbox work appeared. A stayed healthy and B stayed fenced. Secret-free evidence plus a SHA-256 manifest is under `/home/opsadmin/lorawan-ha-evidence/NODERED-OUTBOX-SYNTH-20260829T152713Z`. `NODE_RED_SYNTHETIC_ATOMIC_OUTBOX=PASS` is authoritative. This proves the server application `enqueue/idempotency` path only; real EMU-01 RF/LoRaWAN acceptance remains deferred. Keep this synthetic set only until the immediate Grafana read-path proof, then remove the exact named rows.

The later Grafana commissioning investigation does not change this Node-RED PASS. A fresh Grafana-only fixture `grafana-synthetic-GRAFANA-SYNTH-20260830T000012Z` was subsequently generated through this same exact

deployed Function from a clean reserved-identity baseline. It again produced exactly 1|13|1, and Grafana successfully read it through the commissioned `telemetry_reader` datasource and all four actual dashboard targets. `GRAFANA_SYNTHETIC_FIXTURE_AND_READ_PATH=PASS` is authoritative for the server-only visualization/read boundary. A separate cleanup-only transaction then removed exactly that fixture and proved the event plus reserved all-zero synthetic identity were `0|0|0` on all three PostgreSQL members while Grafana and Node-RED remained healthy. `GRAFANA_SYNTHETIC_CLEANUP_COMPLETE=PASS`. Do not repeat Node-RED synthetic replay commissioning. Although the captured first Grafana API output stopped at a zero-row parser assertion, subsequent three-node forensics proved that the exact synthetic fixture had already been removed by the targeted cleanup path from the combined Grafana/cleanup wrapper. PostgreSQL statistics recorded one uplink delete, thirteen measurement deletes, and one outbox delete; the cluster remained consistent on timeline 3 with zero lag and no telemetry retention jobs. `u1c-03` history contains the exact cleanup SQL and `u1c-01` sudo evidence shows the matching privileged primary-side `psql` invocation at 2026-08-29 15:35:09 UTC. `SYNTHETIC_ROWSET_CLEANUP_ATTRIBUTED=PASS`. Preserve the original hashed `NODE_RED_SYNTHETIC_ATOMIC_OUTBOX=PASS` evidence; do not rerun Node-RED rollout/replay commissioning merely to replace the deleted fixture. If Grafana still needs a non-empty row, create a fresh uniquely named fixture in a separate boundary and keep its later cleanup separate from the Grafana query proof.

Node-RED A activation wrapper stdin-consumption stop - 2026-08-29

The controlled activation wrapper successfully proved Node-RED B fenced, both local dependency TCP routes reachable, started Node-RED A, verified restart count `0`, and proved the editor was bound only to `127.0.0.1:1880`. The wrapper then stopped immediately at Step 6 before printing logical-host resolution output. Treat the printed outer exit `0` as non-authoritative for Steps 6-10: `docker compose exec -T` still keeps stdin attached by default, so when run inside `sudo bash <<'EOF'` it can consume the remaining heredoc, exactly like the earlier SSH stdin-consumption harness defect. Do not restart or recreate A. Continue with read-only checks against the already-running container using `docker exec` without `-i` (or Compose with interactive stdin explicitly disabled), re-prove B remains stopped, inspect startup logs, and then complete HTTP/database/runtime validation.

Node-RED A PgBouncer CA permission root cause - 2026-08-29

Node-RED A started successfully on `u1c-03` with Node-RED B fenced, restart count `0`, healthy HTTP on loopback `127.0.0.1:1880`, and both logical dependency names resolving to `10.104.0.8`. The first runtime database probe then failed before PostgreSQL authentication: Node.js warned that `/run/pgbouncer/ca.crt` could not be loaded because of Permission denied, followed by unable to verify the first certificate. This is not evidence of PgBouncer, PostgreSQL, SCRAM, HAProxy, or certificate-chain failure. The shared Compose file mounted the commissioned host CA directly from `/etc/lorawan-pki/pgbouncer/ca.crt`, whose protected boundary is intentionally `0750 root:postgres` with `0640 root:postgres` files, while the Node-RED container receives only supplementary GID `node-red-secrets`. The bind mount preserves that unreadable ownership.

The repair contract is least-privilege: keep the original PgBouncer PKI permissions unchanged; create `/etc/lorawan-pki/node-red-pgbouncer` as `0750 root:node-red-secrets`; install only a byte-identical copy of the public PgBouncer CA as `/etc/lorawan-pki/node-red-pgbouncer/ca.crt` with `0640 root:node-red-secrets`; and mount that dedicated copy to `/run/pgbouncer/ca.crt`. `NODE_EXTRA_CA_CERTS` and the in-container path do not change. Apply the same public-CA copy and updated shared Compose file to stopped Node-RED B before it is considered failover-ready again. Recreate Node-RED A after the mount correction because `NODE_EXTRA_CA_CERTS` is read at Node.js process startup, then repeat only the failed database/runtime/log/fencing checks; do not redo passed MQTT identity, password rotation, or staging gates.

Node-RED A PgBouncer CA repair and activation PASS - 2026-08-29

Node-RED A on `u1c-03` is now healthy with the corrected PgBouncer CA runtime boundary. A dedicated copy of the public PgBouncer CA was installed at `/etc/lorawan-pki/node-red-pgbouncer/ca.crt` as `root:node-red-secrets` mode `0640`, byte-identical to the commissioned CA SHA-256 `6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb`. The shared Compose mount was updated to use that Node-RED-readable host path while keeping the in-container path `/run/pgbouncer/ca.crt` unchanged.

Node-RED A was recreated successfully, reported healthy with restart count 0, could read the CA through supplementary GID 987, and authenticated through local `pgbouncer.internal.lorawan.com:6432` as `telemetry_writer` to writable `lorawan_telemetry` with `pg_is_in_recovery() = false`. Startup logs show Node-RED 5.0.4 loaded `/data/flows.json`, started flows, and connected by MQTT mTLS as `node-red-ingest` to `mqtt.internal.lorawan.com:18884`. Node-RED B remained fenced/stopped. `NODE_RED_A_PGBOUNCER_CA_REPAIR=PASS` and `NODE_RED_A_ACTIVE=PASS` are authoritative. Before the first real uplink test, apply the same public-CA copy and canonical updated Compose mount to stopped Node-RED B, validate its configuration without starting B, then preserve the single-active state.

Node-RED A/B PgBouncer CA parity PASS - 2026-08-29

Node-RED A remained healthy and active while stopped Node-RED B received the same dedicated Node-RED-readable PgBouncer CA copy and corrected Compose mount. On B, `/etc/lorawan-pki/node-red-pgbouncer/ca.crt` matched the commissioned CA SHA-256 `6773c652aadcc1740e630b3e0ee13ccaff9427df5418e89571b4630584ea4ddb`, the pinned runtime with supplementary GID 987 could read it, Compose validation passed, and B remained stopped with no `:1880` listener. The single-active invariant held throughout. `NODE_RED_A_B_CA_RUNTIME_PARITY=PASS` and `exit 0` are authoritative. The next boundary is one fresh real EMU-01 uplink through the active A path, followed by an exact database query proving one canonical uplink row and the expected normalized measurements; do not start B for this test.

Repository-only v2 gateway-reception provenance candidate - 2026-08-30

Gateway-evidence source work identified a deterministic v2 join already carried by ChirpStack: `integration.UplinkEvent.rxInfo[0]` preserves the original `gw.UplinkRxInfo.uplink_id`, gateway context, RSSI, and SNR, while `txInfo.frequency` supplies the reception frequency. The repository Node-RED candidate now appends nullable `gateway_uplink_id`, `gateway_frequency_hz`, and `gateway_context_base64` to the existing first-reception `gateway_id/RSSI/SNR` fields. The deployed cloud runtime's existing \$1..\$25 semantics are preserved in the candidate: \$25 remains the v1 Fabric selection boolean and the new provenance values are appended as \$26..\$28.

This is **repository source only**. The commissioned live Node-RED A/B flow hash remains the previously recorded `476056c5cff951ff46bb48c2eeb0e153b666c8cdc42eab88532fd3bebbcdc753`, and the live telemetry schema has not been altered by this source tranche. Do not restart/recreate either Node-RED instance just to pick up these fields. Before the first real telemetry-attestation-v2 acceptance event, apply the reviewed nullable telemetry-column migration through the guarded database process, stage the new flow on stopped B first, verify v1 \$25 behavior and single-active fencing, then promote the exact same candidate to A. Missing v2 reception provenance must remain pending in the verifier; never substitute nearest-timestamp matching.

Canonical source: `deployment/technology-guides/10-node-red.md`. For any mutable production-status statement, check the dated baseline and verified live state.